



Jazz Processor User Guide

Version 2.5

Copyright Notice and Proprietary Information

Copyright © 2002 Improv Systems, Inc. All rights reserved. This software and manual are owned by Improv Systems, Inc. and/or its licensors, and may be used only as authorized in the license agreement controlling such use. No part of this publication may be reproduced, transmitted, or translated in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Improv Systems, Inc., or as expressly provided by the license agreement.

Trademarks

Improv, Improv Systems, Improv Systems, Inc., PSA, PSA Platform, and Jazz PSA Platform are registered trademarks or trademarks of Improv Systems, Inc., in the United States and/or other countries. Product and company names mentioned herein are trademarks and/or registered trademarks of their respective companies. Improv System Inc. publication of information regarding any third party's products or services does not constitute Improv System Inc. approval, warranty, or endorsement thereof.

Important Information

Improv Systems, Inc. (Improv) reserves the right to make changes, updates, or corrections to the information in this document and to its products, or to discontinue any product or service at any time without notice. No part of this document may be reproduced or transmitted in any form or by any means for any purpose without the express written permission of Improv. Nothing in this document shall be construed to grant any license or right, either expressly, by implication, estoppel, or otherwise, to any intellectual property right of Improv. All information is provided on an "as is" basis, and to the extent permitted under applicable law, Improv disclaims all warranties, express or implied, including, but not limited to any conditions of quality or any warranties arising by law, statute, usage of trade or course of dealing. To the extent permitted by applicable law, in no event shall Improv be liable for any loss or damage, however caused and on any theory of liability, arising from the use of any information in this document or any error or omission in such information.

Certain applications using semiconductor products may involve potential risks of death, personal injury, or severe property or environmental damage (critical applications). Improv products are not designed, authorized, or warranted to be suitable for use in life-support devices or systems or other critical applications. Inclusion of Improv products in such applications is understood to be fully at the customer's risk. In order to minimize risks associated with the customer's applications, adequate design and operating safeguards must be provided by the customer to minimize inherent or procedural hazards. Improv assumes no liability for applications assistance or customer product design.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to the United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Version 2.5 created September 19, 2005



Contents

List of Figures • vii

List of Tables • ix

Preface • x

Audience.....	x
Document Structure.....	xi
Document Conventions.....	xii
Special Message Conventions	xii
Visual Conventions	xii
Related Publications	xii

Chapter 1

Introducing the Jazz Processor • xiii

Flexibility.....	xiv
Key Features	xv
Benefits.....	xv
DSP Instruction Set.....	xvii

Chapter 2

Input/Output Signal Descriptions • xix

Input/Output Paths	xx
Processor Signal Descriptions	xxi

Chapter 3

Jazz Processor Architecture • xxiii

Registers.....	xxiv
Memory Organization.....	xxix
Data Memory.....	xxix
Instruction Memory.....	xxxi
Data Transfer.....	xxxii
Memory Maps	xxxiii
Addressing Modes.....	xxxiv
Control Unit	xxxvi
Task Queue Unit	xxxviii
Computational Units.....	xxxviii
Arithmetic Logic Unit.....	xl
Multiply and Accumulate Unit.....	xlvi
Shift Unit.....	xlvi
Memory Interface Units.....	xlvi
System Control	xlvi
Hardware Stack	xlvi
Processor Configuration.....	xlvi
Operation.....	xlvi
Implementation.....	xlvi
Instruction Pipeline	xlvi
Interrupts.....	li
Interrupt Queue	li
Control.....	lii

Caveats	liii
---------------	------

Chapter 4

Assembly Language • Iv

Syntax	lvi
Memory Addressing Modes.....	lvi
Direct Addressing Mode	lvii
Indirect Addressing Mode	lvii
Auto-Increment Addressing Mode.....	lvii
Displacement Addressing Mode	lviii
Indexed Addressing Mode	lviii
Circular Buffer Addressing Mode.....	lviii
Bit-Reverse Addressing Mode	lix
Assembly Language Fields	lix
Addition & Subtraction.....	lxx

Chapter 5

Instruction Set Summary • lxxiii

Individual Micro Operations.....	lxxiv
ACA	lxxv
ACZ.....	lxxvi
ADD	lxxvii
ADD16	lxxviii
ADD5	lxxix
ADD8	lxxx
ADDF	lxxxi
ADLUI	lxxxii
AND	lxxxiii
ASR.....	lxxxiv
BR.....	lxxxv
CALL	lxxxvi
CE.....	lxxxvii
CMP	lxxxviii
CMP16	lxxxix
CMP8	xc
CND	xc
*CRI	xcii
CTD.....	xciii
CTS.....	xciv
EQ.....	xcv
ET	xcvi
GT.....	xcvii
GTE	xcviii
HTQ.....	xcix
INTR.....	c
ITQ	ci
JEXP.....	cii
LSL.....	ciii
LSR.....	civ
LSS	cv
LSU	cvi

LT	cvii
LTE.....	cviii
LWD	cix
MASK	cx
MAX.....	cx i
MIN	cxii
MPS.....	cxiii
MPU	cxiv
MPW	cxv
NBR.....	cxvi
NCE	cxvii
NEQ.....	cxviii
OR	cxix
POP.....	cxx
*PR_INI	cxxi
PR_ROT.....	cxxii
PR_WR.....	cxxiii
PUSH.....	cxxiv
R16S	cxxv
R16U	cxxvi
R32	cxxvii
R8S	cxxviii
R8U	cxxix
RET	cxxx
RETE.....	cxxx i
RPT.....	cxxxii
SBR	cxxxiii
SCR	cxxxiv
SCS.....	cxxxv
SCTL	cxxxvi
SEL.....	cxxxvii
SETA.....	cxxxviii
SETWA	cxxxix
SFR.....	cxl
SINTM	cxli
TQ.....	cxlii
TRUE	cxliii
UBR.....	cxliv
UET	cxlv
UGT.....	cxlvi
UGTE	cxlvii
ULT	cxlviii
ULTE.....	cxlix
W16	cl
W32	cli
W8	clii
WPR	cliii
XOR	cliv



List of Figures

Figure 1-1.	Jazz Processor Architecture	xiv
Figure 2-1.	Jazz 2.2 Processor Input/Output Paths	xx
Figure 3-1.	Data Memory Architecture.....	xxx
Figure 3-2.	Data Memory Timing.....	xxx
Figure 3-3.	Instruction Memory Architecture.....	xxx
Figure 3-4.	Instruction Memory Timing	xxx
Figure 3-5.	Instruction Memory Space	xxx
Figure 3-6.	Data Memory Space	xxx
Figure 3-7.	MIU Address Generation Unit	xxx
Figure 3-8.	MIU Address Increment Unit.....	xxx
Figure 3-9.	Computational Unit Architecture	xl
Figure 3-10.	Arithmetic Logic Unit Architecture	xli
Figure 3-11.	Multiply and Accumulate Unit Architecture.....	xlii
Figure 3-12.	Shift Unit Architecture	xliii
Figure 3-13.	Sample Q-Bus	xl
Figure 3-14.	Q-Bus Arbitration Phases.....	xl
Figure 3-15.	Branch Instruction in the Pipeline	xli
Figure 3-16.	Call Immediate Instruction in the Pipeline.....	l
Figure 3-17.	Repeat Instruction in the Pipeline	l
Figure 3-18.	Interrupt Task Queue.....	li
Figure 3-19.	Interrupt Pipeline Response	lii



List of Tables

Table 1-1.	Jazz Processor Configurations	xiv
Table 1-2.	Key Features	xvi
Table 2-1.	Input/Output Ports.....	xxi
Table 3-2.	Address Registers.....	xxv
Table 3-1.	Global Registers.....	xxv
Table 3-3.	Constant Registers	xxvi
Table 3-4.	Control Register MOPs.....	xxviii
Table 3-5.	Status Register MOPs	xxviii
Table 3-6.	Control Unit MOPs	xxxvii
Table 3-7.	Task Queue MOPs	xxxviii
Table 3-8.	Computational Unit MOPs	xxxix
Table 3-9.	Memory Interface Unit MOPs	xliv
Table 4-1.	Assembly Language Syntax.....	lvi
Table 4-2.	Memory Units	lvii
Table 4-3.	Unit Registers.....	lxvii
Table 4-4.	Shift Unit Saturation	lxix
Table 4-5.	Conditions	lxx
Table 4-6.	Arithmetic Conditions.....	lxxi
Table 5-1.	Condition Mask Values.....	cxxxiv



Preface

Welcome to the *Jazz Processor User Guide*. This document provides the information you need to understand and use the Jazz RTL processor. The processor is an integral part of the Improv solution for developing today's applications.

The Jazz processor is a fixed-point Digital Signal Processor (DSP) in the Jazz PSA family. This manual defines the architecture of the Jazz processor, and how to use the instruction set to develop hardware and software applications on the processor.

In This Preface:

Document Conventions • xii

Related Publications • xii



Audience

This manual is intended for use by systems engineers who develop applications using Improv Systems' Jazz PSA system-on-chip technology.

Document Structure

This document is divided into five chapters.

Chapter 1, Introducing the Jazz Processor

In Chapter 1, you will be introduced to the flexibility and unique features of the Jazz processor, and the functions supported by the DSP instruction set.

Chapter 2, Input/Output Signal Descriptions

In Chapter 2, you will learn about the input/output paths for the processor and the signal descriptions.

Chapter 3, Jazz Processor Architecture

In Chapter 3, you will learn about the architectural design of the processor, the length of on-chip memory, the MIU ports available, the internal hardware structure, the memory organization, and the functions you can perform using the processor.

Chapter 4, Assembly Language

In Chapter 4, you will learn about the assembly language structure and syntax used to develop programs on the processor. All fields listed in each micro operation in Chapter 5 are also described.

Chapter 5, Instruction Set Summary

In Chapter 5, you will learn about the organization of the instruction set, and the encoding structure. Each micro operation within the instruction set is also described in detail.

Document Conventions

This guide uses a variety of formats to highlight specific types of information.

Special Message Conventions

Special message conventions refer to noteworthy information, and include a symbol for quick identification:



Note: *Identifies information of importance or special interest.*

Visual Conventions

Visual conventions refer to specific types of information, formatted in a particular manner, as outlined in the table below.

<code>courier</code>	Identifies syntax statements, onscreen text, and path, file, drive, directory, database, table names.
<code><courier></code>	Identifies variable names.
<code>courier bold</code>	Identifies text you must type to complete a procedure.
<i>italics</i>	Identifies document and chapter titles, special words or phrases used for the first time, words of emphasis, domain names, and email addresses.
Initial Caps	Identifies window, menu, dialog, command, button, option, tab, keyboard, and product specific names.
ALL CAPS	Identifies acronyms and abbreviations.
[]	Identifies an optional item in a syntax statement.
{ }	Identifies an optional syntax statement item that can be repeated as necessary.

Related Publications

Although the primary focus of this guide is the Jazz processor, references must be made to the other technologies in the Improv System solution. The Improv publications related to this guide are:

- *Jazz PSA™ Composer GUI*,
- *Jazz Processor Generator User Guide, Version 2.2.*



Chapter 1 Introducing the Jazz Processor

The Jazz processor is a leading-edge Digital Signal Processing (DSP) technology that can be configured for application-specific designs. The unique capabilities of the Jazz processor provide users with the ability to incorporate specialized hand crafted, gate-level descriptions within the data path of the Jazz programmable processor.

This chapter provides an introduction to the features and functions of the Jazz processor.

In this chapter, you will learn about:

- the way in which the Jazz processor can be configured to match user requirements,
- the key features of the Jazz processor that distinguish it from other DSP products,
- the typical applications that the Jazz processor technology can support,
- the contents of the DSP instruction set.

The Jazz processor is designed using an architecture that can be configured to specific requirements. This feature, combined with the Very Long Instruction Word (VLIW) architecture and special Digital Signal Processor (DSP) instruction set, provides each Jazz processor with scalable performance. In excess of two billion operations can be executed per second. In addition to the standard complex DSP hardware computational functions provided by other DSP processors, Jazz processors can be configured to incorporate application specific, designer-defined computational functions. The flexibility of the Jazz processor hardware architecture and the Notation toolset offer application design engineers development capabilities that were previously unavailable in other DSPs.

In This Chapter:

Flexibility • xiv

Key Features • xv

DSP Instruction Set • xvii



Table 1-1 summarizes a few examples of basic Jazz processor configurations. The number of slots within a processor can be configured.

Table 1-1. Jazz Processor Configurations

Configuration Name	CUs	MIUs	Slots
Jazz2010	3	2	2
Jazz2020	6	2	2
Jazz2040	10	4	7

Flexibility

The Jazz processor provides the ability to scale computational parallelism to meet the performance requirements of the application. This is accomplished by expanding the data path of the processor with additional execution units. Flexibility is further enhanced by enabling users to define and program multi-processor implementations within the same methodology. The Jazz processor leverages an advanced design methodology to permit the configuration and extension of the architectural features of the Jazz processor to better suit the application. Improv's product solutions allow the application developer to drive the hardware definition of the programmable processor.

Figure 1-1 illustrates an overview of the Jazz processor architecture.

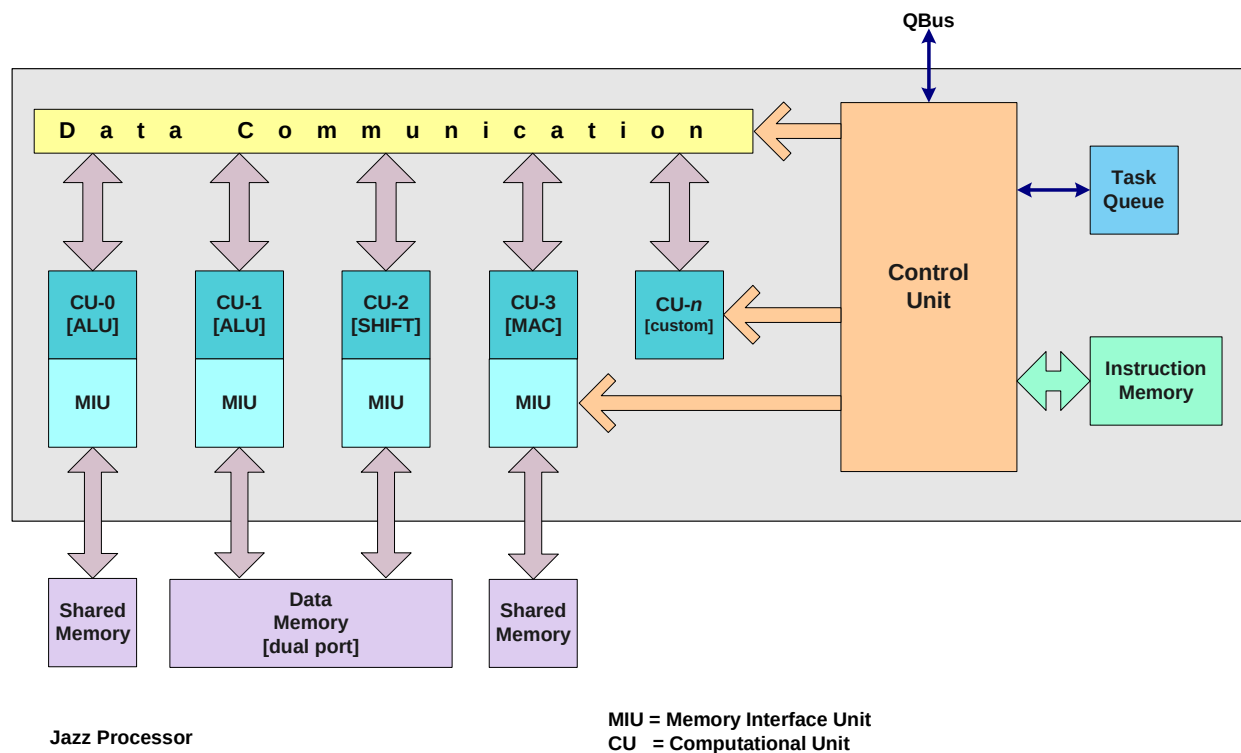


Figure 1-1. Jazz Processor Architecture

Key Features

The Improv architecture permits the integration of a multi-Jazz processor platform with legacy processor platforms and peripherals. These are identified as “Integration Blocks”, and can consist of very simple or very complex logic or functional blocks containing processors, random logic, data channels, and so on. The Jazz processor communicates with these integration blocks through shared memory architectures, providing a straightforward method to communicate with Jazz processors in the system implementation.

- Multiple Jazz processors are configured in a structure that can be programmed to perform application-specific functions.
- Integration blocks (or I-Blocks) can be added to provide specific interface logic functions around the structure.
- Processor-to-processor data communication is handled through direct on-chip data memories.
- All processors attach to a single Q-Bus that enables queuing of tasks.

The Improv architecture scales from a single, unique Jazz DSP processor configuration, to a system-level platform implementation. The resultant implementation consists of many unique Jazz processor configurations in an interconnected structure. This structure is defined by shared memory maps between processors.

Jazz processors are tightly coupled for control purposes, with a proprietary bus called the Q-bus. The Q-bus acts as a unified task management and control bus to synchronize many Jazz processors in a system platform.

Benefits

The benefits of the Jazz processor are grouped by primary feature, and are summarized below.

- Designer-Defined Computational Units (DDCUs),
- overlapping Computational Units (CUs), Memory Interface Units (MIUs), and DDCUs,
- wait-state generation,
- bus arbitration,
- internal and/or external instruction memory,
- low and high priority tasks,
- interrupt tasks,
- VLIW processor architecture:
 - parallel resource execution,
 - up to 34 RISC-like operations per instruction cycle,
 - two-stage pipeline,
 - multiprocessor ready,
 - zero glue through use of the Q-Bus interface,
 - available in 32- and 16-bit data path versions,
 - single-cycle MAC unit,
 - instruction compression.

- Low Power Support:
 - IDLE Mode,
 - static state held,
 - SLEEP mode,
 - glitch-free, 98 percent clock shut down.
- DSP Conscious Addressing Modes:
 - index,
 - bit reversed,
 - circular buffer.
- A Design That Can Be Fully Synthesized:
 - no special physical library requirements,
 - no register files,
 - synchronous clocked operation,
 - static clock timing.
- Full Design Methodology Support:
 - delivers Verilog RTL,
 - automatic synthesis script generation,
 - automatic static timing analysis script generation,
 - verification/regression test suite generation.

Typical Applications

The Jazz processor architecture supports a variety of applications across a number of industry sectors. *Table 1-2* lists a few of the numerous applications that the Jazz processor architecture supports within several technology areas.

Table 1-2. Key Features

Segment	Equipment	Key Applications	Key DSP
Residential Gateway	Broadband Modems Voice/ Data Gateways Set Top Box	Voice- Over- Packet Streaming Media Security Advanced Video/ Audio Wireless LAN Wireless/DSL/Cable Modem	Voice Compression Echo Cancellation Encryption Modulation
Enterprise CPE	Integrated Access Devices IP Phone	Voice-Over-Packet (DSL, T1) Teleconferencing Security	Voice Compression Echo Cancellation Encryption Modulation Packet Classification
Network Infrastructure	High- Speed Switching/Routing	QoS Data Switching	Packet Classification High-Speed FEC Modulation
3G Wireless/PDA	Base Station Handset Integrated PDA	Streaming Media Voice Processing Wireless Internet/ Connectivity Advanced Audio/ Video	Voice Compression Echo Cancellation Encryption LBR Video/Audio MP3 Audio Modulation
Imaging & Appliance Market	Printer/Copier Digital Camera: Still/Video DTV/DVD Digital Video & Audio	Image Processing Image/ Video/Audio Recording	Image Processing Image/Video Compression LBR Speech/Video/Audio High-End Video/Audio

DSP Instruction Set

The Jazz processor is a VLIW (Very Long Instruction Word) processor that includes the following components:

- a Control Unit,
- a Task Queue,
- a Data Communication Module,
- multiple Memory Interface Units (MIUs) ,
- multiple Computational Units (CUs).

The type of operations that can be performed by each component are defined below.

Control Unit

The control unit facilitates the following type of operations:

- conditional and unconditional branch,
- instruction repeat,
- stack,
- counter,
- interrupt masking,
- condition testing,
- power saving.

Task Queue

The task queue unit facilitates the following type of operations:

- platform communication,
- program initiation through task fires.

Memory Interface Unit

Memory Interface Units (MIUs) facilitate the following type of operations:

- address registering,
- post-increment,
- memory byte, half word, and word Load and Store.

Computational Unit

Computational Units (CUs) facilitate the following type of operations:

- designer-defined,
- addition,
- relational,
- logical,
- exponent extraction,
- condition test,
- multiply and accumulate¹,
- barrel-shift².

¹ Single-cycle pipelined.

² Where the barrel-shift is a multiplier and/or divisor of the $2^{\text{shift_amount}}$.



Chapter 2 Input/Output Signal Descriptions

This chapter describes the input and output (I/O) signals within the Jazz processor. The fields are listed alphabetically in *Table 2-1*.

The diagram on the next page illustrates the input and output paths in a typical Jazz processor for the following units:

- Computational Units (CUs),
- Designer-Defined Computational Units (DDCUs),
- Memory Interface Units (MIUs);
 - Read-Only Double Pumped,
 - Read/Write Single Pumped,
 - Read/Write Double-Pumped.

In This Chapter:

Input/Output Paths • xx

Processor Signal Descriptions • xxi



Input/Output Paths

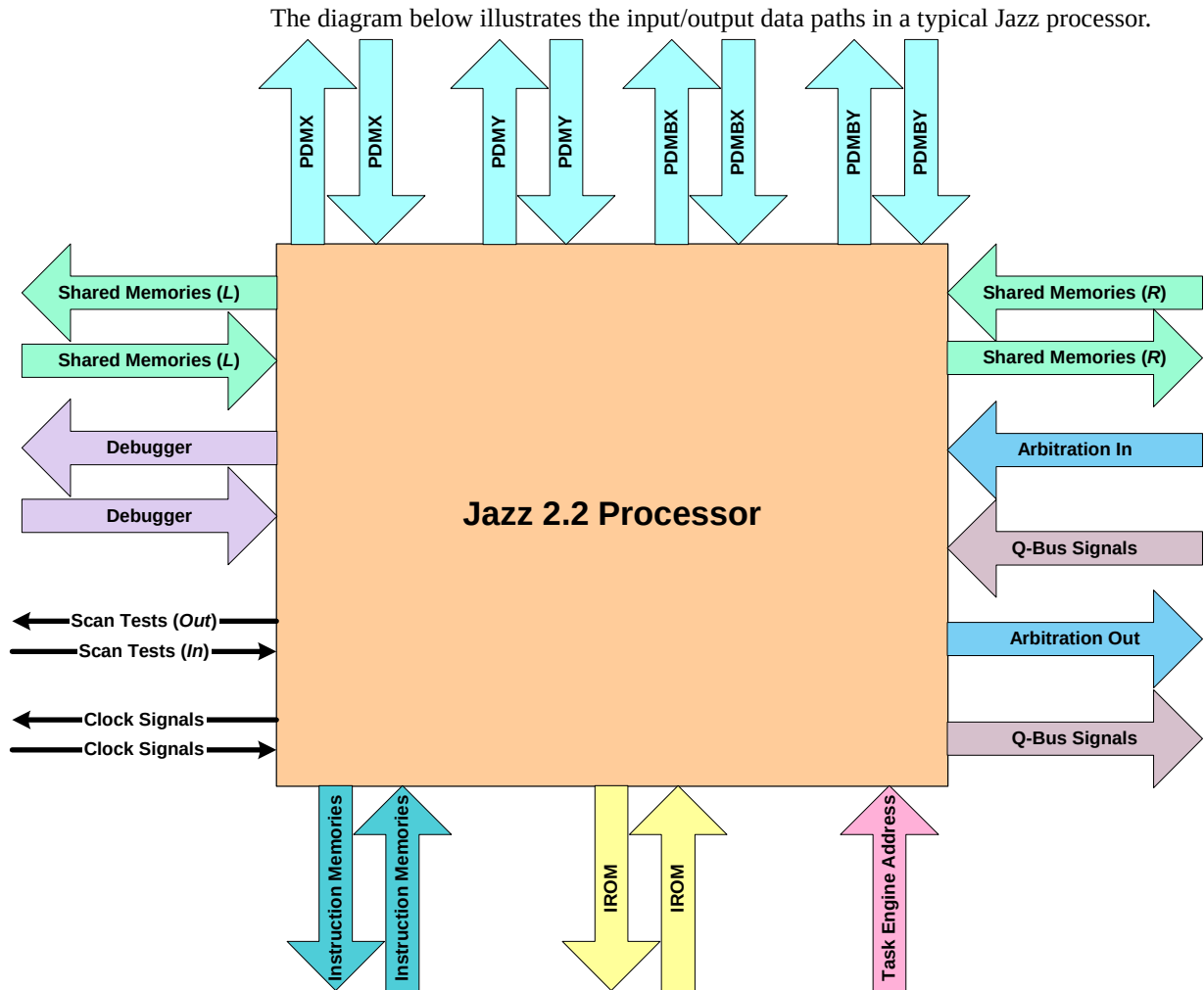


Figure 2-1. Jazz 2.2 Processor Input/Output Paths

Processor Signal Descriptions

Table 2-1 defines the Input/Output (I/O) ports within a typical Jazz processor architecture. The table is organized alphabetically by signal name, and includes the direction (input or output) of the signal, the size (in bits), and a description of the signal.

The Jazz processors can be configured to meet specific performance requirements. For this reason, some of the ports listed in *Table 2-1* are defined using a generic labelling format, and do not necessarily reflect the precise nomenclature applied to the processor in a specific implementation kit.

The names in the table below enclosed in brackets (<>) conform to the following names used in the given processor configuration:

<processor> the name of the processor (e.g., Jazz_32),
<uname> the instance name.

Table 2-1. Input/Output Ports

Signal	I/O	Size	Description
<processor>_miu_<uname>_memaddr	O	14	MIU memory read address.
<processor>_miu_<uname>_memcs_n	O	4	MIU memory chip select.
<processor>_miu_<uname>_memwdata	O	32	MIU memory write data port (optional).
<processor>_miu_<uname>_memwe_n	O	1	MIU memory write enable (optional).
<processor>_im_imem_addr	O	12	Instruction RAM address port.
<processor>_im_imem_cs_n	O	1	Instruction RAM chip select.
<processor>_im_imem_wdata	O	IM_WIDTH	Instruction RAM write data.
<processor>_im_imem_we_n	O	1	Instruction RAM write enable.
<processor>_im_irom_addr	O	9	Instruction ROM address port.
<processor>_im_irom_cs_n	O	1	Irom chip select.
<processor>_taskq_arbout	O	1	Q-Bus arbitration output signal.
<processor>_taskq_qbusout_l	O	34	Q-Bus to left.
<processor>_taskq_qbusout_r	O	34	Q-Bus to right.
arbin	I	1	Q-Bus arbitration input signal.
clk_nosleep	O	1	1x output clock.
imem_rdata	I	IM_WIDTH	Instruction RAM data port.
imem_ready	I	1	Instruction memory wait signal.
irom_rdata	I	IM_WIDTH	Instruction ROM data port.
memrdata	I	32	MIU memory read data port.
qbusin_l	I	34	Q-Bus from left.
qbusin_r	I	34	Q-Bus from right.
scanen	I	1	Scan enable.
scanin	I	1	Scan data input.

Table 2-1. Input/Output Ports

Signal	I/O	Size	Description
scanin2-N	I	N-1	Scan data input for chains 2-N (optional).
scanout	O	1	Scan data output.
scanout2-N	O	N-1	Scan data output for chains 2-N (optional).
sys_clkmem	I	1	2x memory records external memories and generates internal clocks.
sys_rst_n	I	1	Active low reset signal.
teaddr	I	4	Task engine address.
testmode	I	1	Force chip into testmode – clock control.
jice_pdhi	I		Platform debug halt in.
jice_pdho	O		Platform debug halt out.
jazz_mv_x_tdo	O		Processor test data out.
jdac_tdo	O		Memory test data in.
tdi	I		Processor test data in.
sys_tck	I		Test clock.
update_dr	I		Update data register.
shift_dr	I		Shift data register.
jdac_cs	I		Jazz debug access chip select.
<uname>_tdo	I		Memory unit test data out.



Chapter 3 Jazz Processor Architecture

The Jazz processor is based on the Very Long Instruction Word (VLIW) architecture with parallel resource execution. This flexible architecture can be configured for specific user requirements, and includes the following features:

- a task queue architecture that enables efficient context switching;
- *readied* data before a task is initiated;
- non-preemptive tasks (except interrupts);
- a two-stage pipeline for instruction fetches;
- *on-the-fly* decompressed instructions;
- low power support for IDLE and SLEEP modes;
- DSP-conscious addressing mode (indexed, BR, CB).

The Jazz processor is composed of the following units:

- Core Units,
- Computational Units (CUs),
- Memory Interface Units (MIUs).

The following components can be configured using the Jazz Composer GUI:

- data width,
- computational units,
- memory interface units,
- registers,
- queues.

In This Chapter:

Registers • xxiv

Memory Organization • xxix

Control Unit • xxxvi

Task Queue Unit • xxxviii

Computational Units • xxxviii

Memory Interface Units • xlv

System Control • xlv

Interrupts • li



Registers

The Jazz processor contains a variety of registers; some of which are grouped into pairs. This section defines the following registers in the Jazz processor, and the values for each one:

- *Global Registers,*
- *Address Registers,*
- *Constant Registers,*
- *Program Register,*
- *Branch Address Register,*
- *Condition Register,*
- *Counter Registers,*
- *Task Queue Parameter Register,*
- *Interrupt Parameter Register,*
- *Control Register,*
- *Status Register.*

Global Registers

Global registers are the result registers in the CUs and MIUs within a processor. These registers are 32-bits wide, and are used to store the 32- and 64-bit results of these units.

For 64-bit operations in the MAC, Shift and some Designer-Defined Computational Units (DDCUs), the results are stored in a global register pair. This pair is composed of two sequential 32-bit global registers. The odd numbered register is used to store the most significant word of the result. The even numbered register is used to store the least significant word of the result (e.g., `sh0.r1:sh0.r0 = 64-bit result`).

CUs, including DDCUs, may be configured for up to 22 result registers in increments of two (e.g., two, four, six, ... 22). The number and type of unit can also be configured. The number and name of these registers varies across configurations. These registers are identified by unit name and register number within a given unit. MAC unit types contain both multiply and accumulate result registers; therefore, the accumulate result registers are identified by an *ac* prefix to the register number.

MIUs may be configured for one, two, four, eight, 12, or 16 (or zero) result registers. The number and type of unit can also be configured. The number and name of these registers varies across configurations. These registers are identified by unit name and port, and register number within a given unit.

Our standard instruction example contains four MIUs (`pdmx`, `pdmy`, `sml` and `smr`). Each MIU and five CUs are configured with one result register (`al0`, `al1`, `al2`, `sh0`, `mp0`). The CUs are configured with four result registers.

Table 3-1 defines the global registers in the standard instruction example.

Table 3-1. Global Registers

MIU Result Registers	pdm.x.r0 sml.x.r0	pdm.y.r0 smr.p.r0		
CU Result Registers	al0.r0 al1.r0 al2.r0 al3.r0 sh0.r0 mp0.r0 mp0.ac0	al0.r1 al1.r1 al2.r1 al3.r1 sh0.r1 mp0.r1 mp0.ac1	al0.r2 al1.r2 al2.r2 al3.r2 sh0.r2 mp0.r2 mp0.ac2	al0.r3 al1.r3 al2.r3 al3.r3 sh0.r3 mp0.r3 mp0.ac3

Address Registers

Address registers are the registers within MIUs that are used to calculate and store memory addresses. These registers are 16 bits wide.

There are two, four, eight, 16, and 32 address registers for each MIU. The MIUs can be configured as read only or read/write. These registers are identified by unit name and port, and address register number within a given unit.

Table 3-2 defines the address registers in a standard processor configuration.

Table 3-2. Address Registers

PDMX	pdm.x.ar0 pdm.x.ar4	pdm.x.ar1 pdm.x.ar5	pdm.x.ar2 pdm.x.ar6	pdm.x.ar3 pdm.x.ar7
PDMY	pdm.y.ar0	pdm.y.ar1	pdm.y.ar2	pdm.y.ar3
SML	sml.x.ar0 sml.x.ar4	sml.x.ar1 sml.x.ar5	sml.x.ar2 sml.x.ar6	sml.x.ar3 sml.x.ar7
SMR	smr.p.ar0 smr.p.ar4	smr.p.ar1 smr.p.ar5	smr.p.ar2 smr.p.ar6	smr.p.ar3 smr.p.ar7

Constant Registers

Constant registers are registers within the control unit of a processor. These registers are all 16 bits wide and are used to store the instruction immediate values.

Numerous processor operations can occur simultaneously in a VLIW processor architecture. Therefore, a variety of instruction immediate values must be accessed at the same time. In the Jazz processor (Version 2.2), only one 16-bit constant value is immediately available in the standard instruction. Other constant values are selected from the data and address constant registers.

The Shift CU and some DDCUs may access the immediate constant and data constant registers. MIUs may access either the immediate, data, or address constant registers. Multiple constant values are updated within the constant registers by the Jazz processor Constant Register Immediate (CRI) special instruction.

The Jazz processor includes one immediate constant value, up to 32 stored data constant values, and up to 32 stored address constant values. These values may vary across processor configurations.

Table 3-3 defines the values for the constant registers in a standard processor configuration.

Table 3-3. Constant Registers

Immediate	ctl.cst0			
Data	ctl.cst1	ctl.cst2	ctl.cst3	ctl.cst4
	ctl.cst5	ctl.cst6	ctl.cst7	
Address	ctl.addrcst1	ctl.addrcst2	ctl.addrcst3	
	ctl.addrcst4	ctl.addrcst5	ctl.addrcst6	
	ctl.addrcst7			

Program Register

The program register is the register within a processor that is used to program instruction memory and initialize data memory.

The program register interfaces between the various 32-bit processor units and the VLIW memory. Therefore, the width of the program register is equal to the width of the VLIW.

For 32-bit operations that use the program register as a data source, the least significant 32-bit word of the program register is used as the source, and the entire program register is shifted to the right by 32 bits.

For 32-bit program register storage operations, the values are stored in the least significant 32-bit word. The previous contents of the program register are shifted to the left by 32 bits.

For instruction memory programming operations, the entire contents of the program register are used as the data source. The least significant bit of the program register is mapped to the least significant bit of the VLIW.

For Program Register Initialization (PR_INI) operations, the multiple 32-bit values within the VLIW are stored in the program register.

Different bits of the program register are used for a variety of operations in the Jazz processor. In each case, the common reference to the program register is given by the control unit and program register names as follows:

ctl.pr

Branch Address Register

The branch address register is the 16-bit register that stores the target address used in branch operations. This register holds the result of the Set Branch Register (SBR) operation, and is not visible to any other unit within the processor.

The branch address register name is as follows:

ctl.br

Condition Register

The condition register is the one-bit register that stores the result of a conditional test operation. This register is used by conditional branch operations and is not visible to any other unit within the processor.

The condition register name is as follows:

`ctl.cnd`

Counter Registers

Counter registers are the 16-bit registers that store the result of counter operations. There are four counter registers in all processor configurations. These registers are used to generate counter terminal counts for condition register operations and are not visible to any other unit within the processor.

The counter register names are as follows:

`ctl.cnt0`

`ctl.cnt1`

`ctl.cnt2`

`ctl.cnt3`

Task Queue Parameter Register

The 16-bit Task Queue Parameter register contains the task parameter of the currently executing standard or high priority task.

The task parameter register name is as follows:

`ctl.task_param`

Interrupt Parameter Register

The 16-bit Interrupt Task Queue Parameter register contains the task parameter of the currently executing interrupt task.

The interrupt parameter register name is as follows:

`ctl.intr_param`

Control Register

The eight-bit Control register is used to control the Task Queue hardware. The Control register name is as follows:

`ctl.ctl_reg`

The Control register bit encoding is defined as follows:

7	6	5	4	3	2	1	0
EITQ	EPTQ	BITQ	BPTQ	BSTQ	WUITQ	WUPTQ	WUSTQ

Table 3-4 defines the Micro Operations (MOPs) in the Status register.

Table 3-4. Control Register MOPs

MOP	Description
WUSTQ	Wake-Up Due to a Standard Task Queue Event
WUPTQ	Wake-Up Due to a Priority Task Queue Event
WUITQ	Wake-Up Due to an Interrupt Task Queue Event
BSTQ	Enable Standard Task Queue for Broadcast Events
BPTQ	Enable Priority Task Queue for Broadcast Events
BITQ	Enable Interrupt Task Queue for Broadcast Events
EPTQ	Enable Priority Task Queue for Processor Events
EITQ	Enable Interrupt Task Queue for Processor Events

Status Register

The 16-bit Status register is used to identify the state of the processor. The Status register name is as follows:

```
ctl.status_reg
```

The Status register bit encoding is defined as follows:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	ITQO	PTQO	STQO		ITQP	PTQP	STQP			ID5	ID4	ID3	ID2	ID1	ID0

Table 3-5 defines the MOPs in the Status register.

Table 3-5. Status Register MOPs

MOP	Description
ID [5 : 0]	Processor ID
STQP	Standard Task Queue Event is Pending
PTQP	Priority Task Queue Event is Pending
ITQP	Interrupt Task Queue Event is Pending
STQO	Standard Task Queue Overflowed (cleared by read)
PTQO	Priority Task Queue Overflowed (cleared by read)
ITQO	Interrupt Task Queue Overflowed (cleared by read)

Memory Organization

Jazz processors use an extension of the Harvard architecture, where data and instruction memory reside in separate spaces. The data memory is further segregated into private and shared spaces.

All Jazz processors require at least one private data memory space. Configurations can include up to two private data memory spaces that are used by Jazz processors to store local information, and may only be accessed by the local processor.

All Jazz processors require at least one shared data memory space. Configurations can include up to two shared data memory spaces that are used by Jazz processors to store local information, and pass data to and/or from another Jazz processor or peripheral device.

All of the data memory spaces support up to 64K bytes of data memory.

Jazz processors have a single instruction memory space that can be configured for up to 64K Platform System Architecture (PSA) words. Each Jazz processor can be configured with up to three physical interfaces to this instruction memory space. Interface partition of this space can also be configured.

All memory spaces are independent and can be accessed simultaneously.

Data Memory

Data memory can include up to four spaces x 64KB of on- or off-chip RAM or ROM. Data memory may be single or dual ported.

Each Jazz processor can have one or two private data memory spaces (pdm and pdmB). Each Private data memory space can have one or two physical data memory ports (*x* and *y*). Each Jazz processor can have one or two shared data memory spaces (sml and smr). Each Shared data memory space has one physical data memory port (for memory space sml, the port interface is *x*; for memory space smr, the port interface is *p*). Each processor can have a maximum of six physical data memory ports and four data memory spaces as follows:

1. Private Data Memory Space (pdm.x, pdm.y),
2. Private Data Memory Space (pdmB.x, pdmB.y),
3. Shared Data Memory Space (sml.x),
4. Shared Data Memory Space (smr.p).

Figure 3-1 illustrates an overview of data memory architecture.

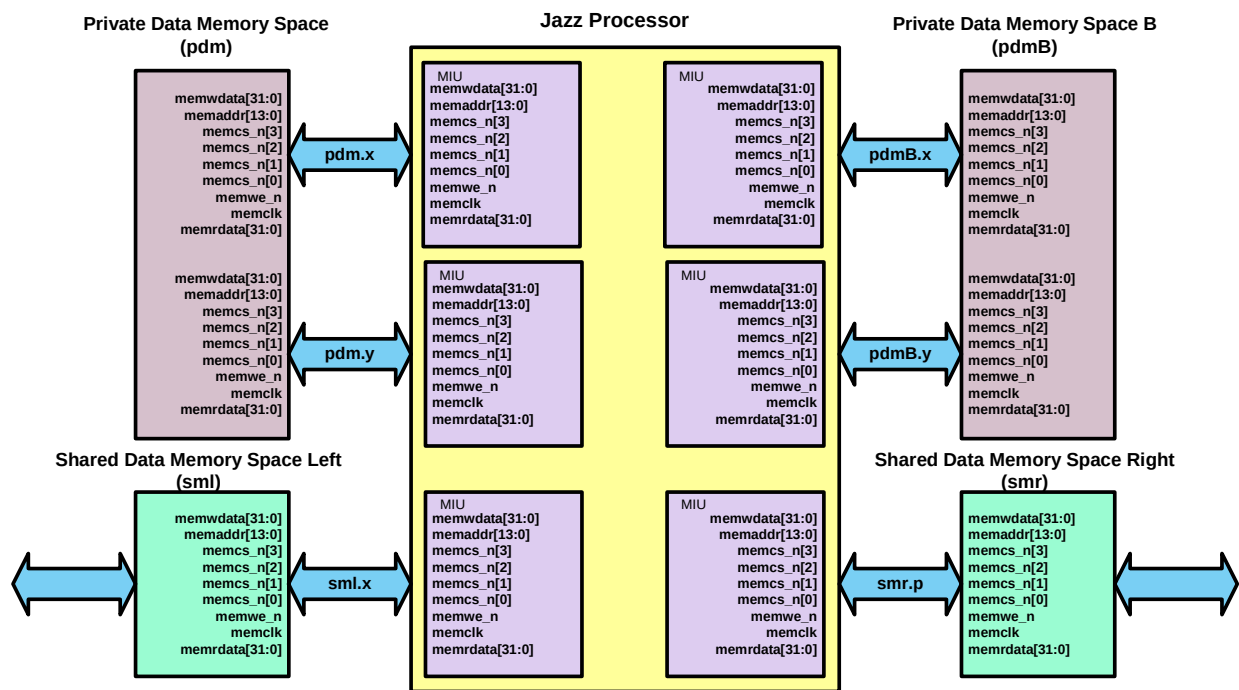


Figure 3-1. Data Memory Architecture

Figure 3-2 illustrates data memory timing.

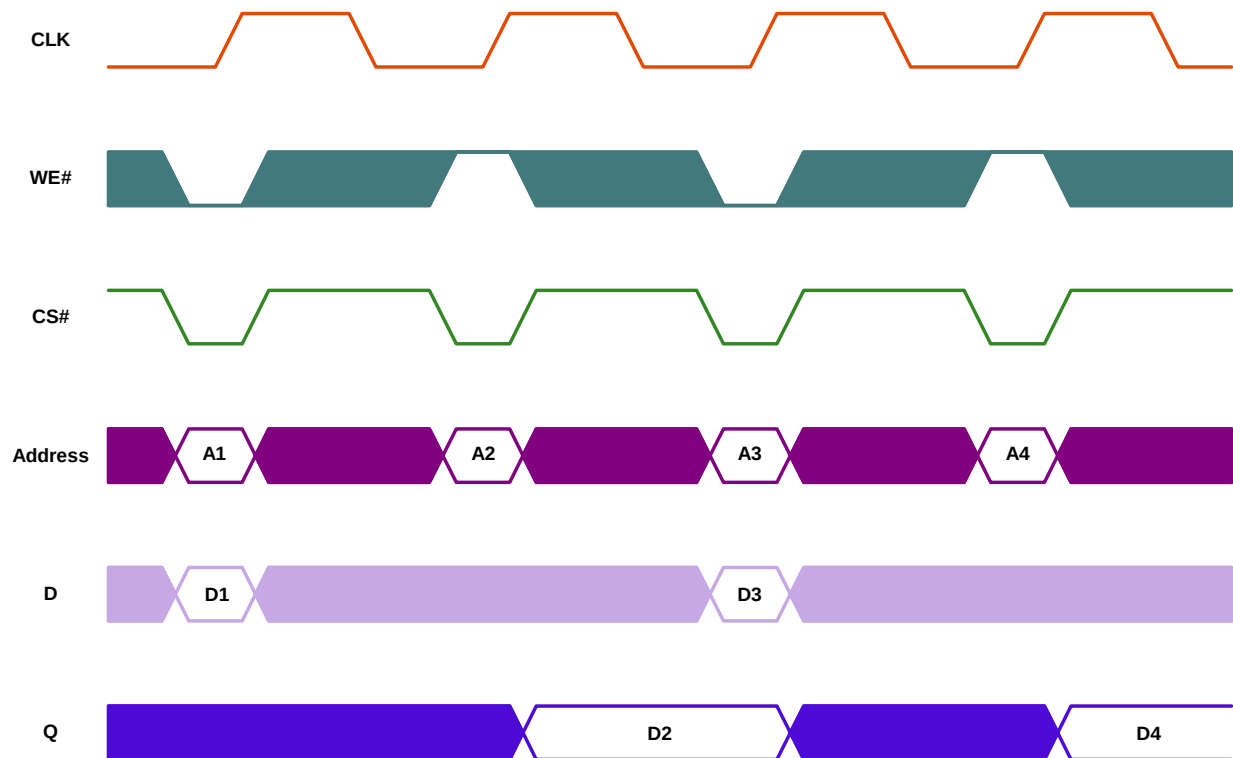


Figure 3-2. Data Memory Timing

Instruction Memory

Instruction memory can comprise up to one space x 64K PSA words of on- or off-chip RAM or ROM.

All instruction memory interfaces execute at the same frequency as the core clock (clk1x) in the processor.

Figure 3-3 illustrates an overview of instruction memory architecture.

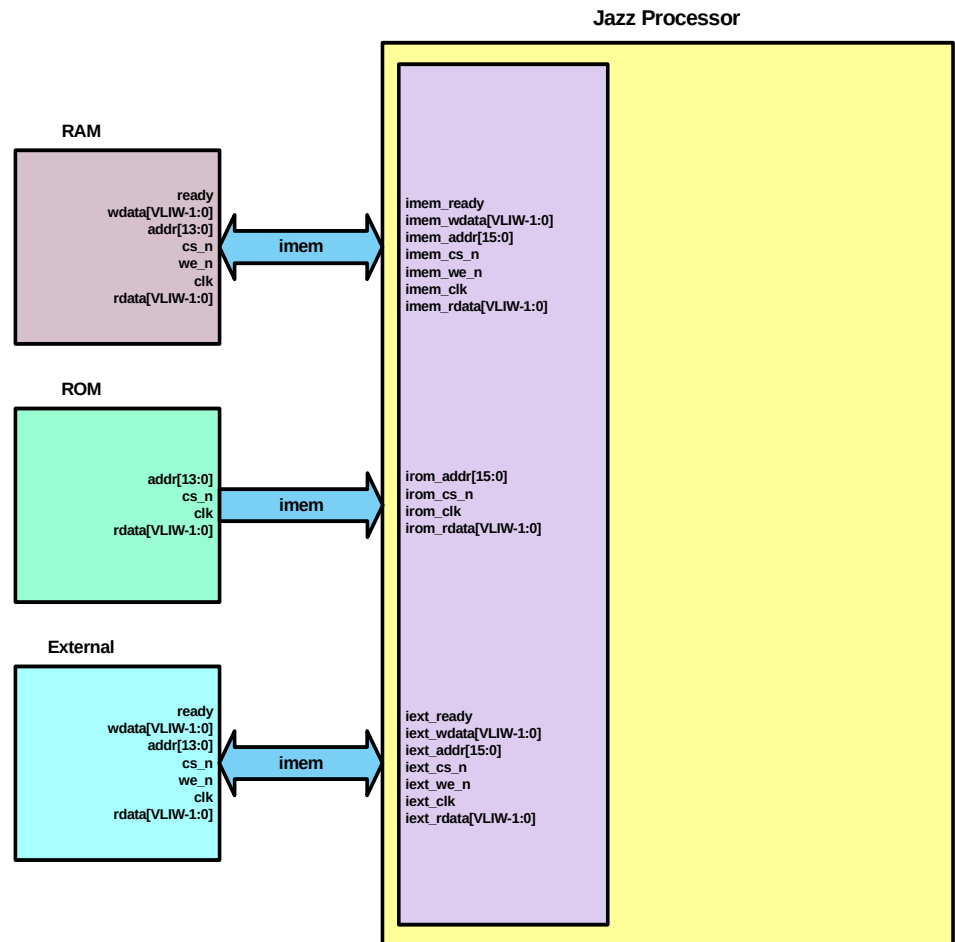


Figure 3-3. Instruction Memory Architecture

Figure 3-4 illustrates instruction memory timing.

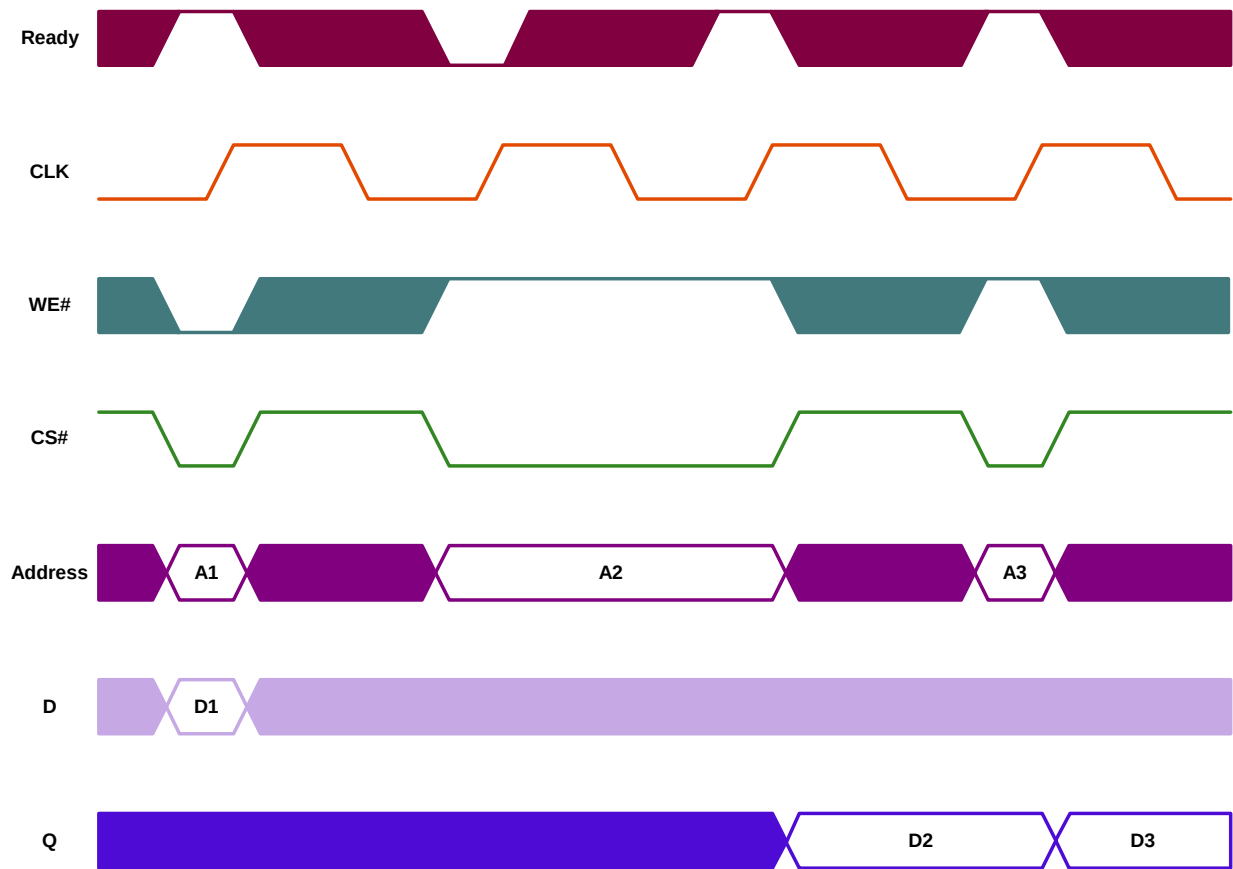


Figure 3-4. Instruction Memory Timing

Data Transfer

Data transfer for both data and instruction memory space is accomplished through a synchronous interface. When the chip select is asserted (active low) on the rising clock edge, data is transferred to and/or from the Jazz processor. When the write enable is asserted (active low), data is moved from the processor (store). When the write enable is de-asserted, data is transferred into the processor (load) on the next rising clock edge.

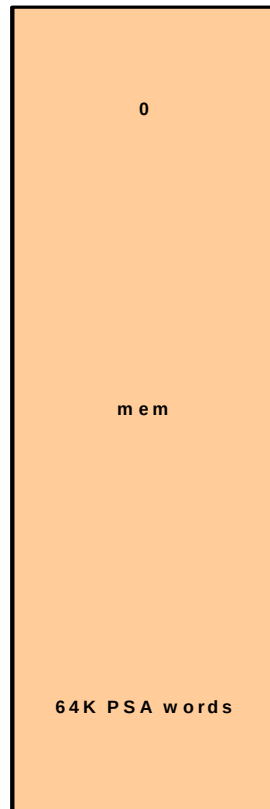
For instruction memory space, the slave memory device can delay the data transfer through the IMEM interface by de-asserting the *imem_ready* signal (active high). The Jazz processor continues to assert the chip select, address, and data buses until the *imem_ready* signal is asserted on a rising clock-edge. The IEXT interface also includes an *iext_ready* signal with similar functionality. The IROM interface does not include this mechanism to stall the processor; therefore, the slave device connected to this interface must be capable of executing at the instruction memory clock rate (core clock or clk1x rate).

Memory Maps

The Jazz processor provides one instruction memory space and up to four data memory spaces. Instruction memory space can be divided into three partitions. Each partition can be configured with the following restrictions:

1. Each partition must contain a minimum of 1024 PSA words;
2. The start address of a partition must be a multiple of the partition size.

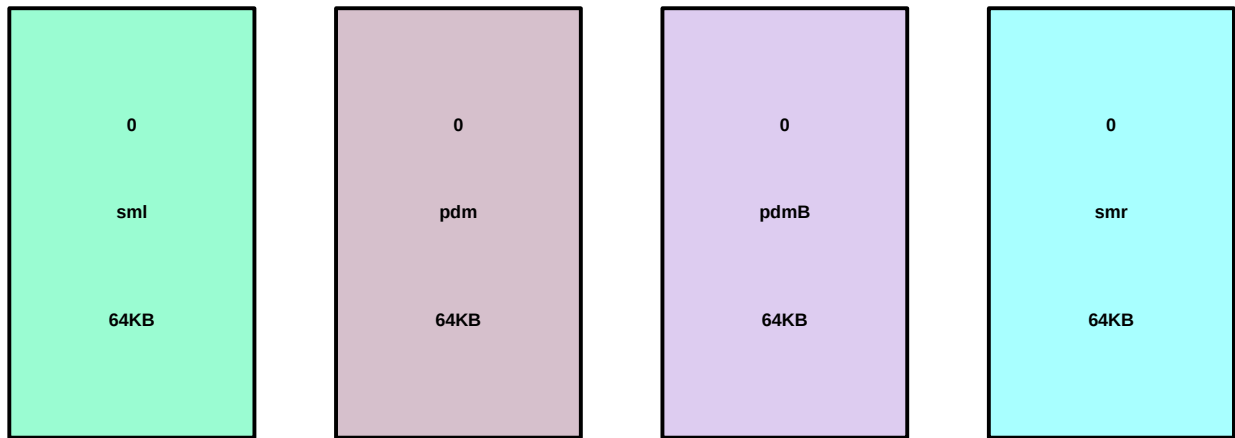
Figure 3-5 illustrates the partition of instruction memory space.



One Data Memory Space for Up to 64 K PSA Words

Figure 3-5. Instruction Memory Space

Figure 3-6 illustrates the partition of data memory space.



Up to Four Data Memory Spaces of Up to 256 KB

Figure 3-6. Data Memory Space

Addressing Modes

The Jazz processor includes the following four types of addressing modes:

- direct,
- indirect,
- auto-increment data memory,
- displacement,
- indexed,
- circular buffer,
- bit-reversed.

Each data MIU can perform the following two types of addressing operations:

- pipelined displacement
- indexed addressing.

MIUs can perform the four types of addressing operations independently for all memory functions. When multiple memory operations are performed simultaneously, the address modes for each memory operation remain independent.

Data memory can be addressed by byte (8-bit) and encompasses a 16-bit address space. Therefore, the maximum data memory size is 64K bytes (16K, 32-bit words). The 32-bit and eight-bit memory accesses (reads and writes) can be performed with data memories. All memory accesses must be aligned according to the size of the data being accessed. For example, on a 16-bit access the least significant address bit must be zero. On a 32-bit access the two least significant address bits must be zero. Addresses follow the Endian convention whereby the byte with the lowest address in a word or half-word access is the least significant eight bits of the word or half-word.

Instruction memory can be addressed by instruction word, and resides on an independent 16-bit address space. The maximum instruction memory size is 64K instructions.

Figure 3-7 illustrates the function of the MIU address generation unit.

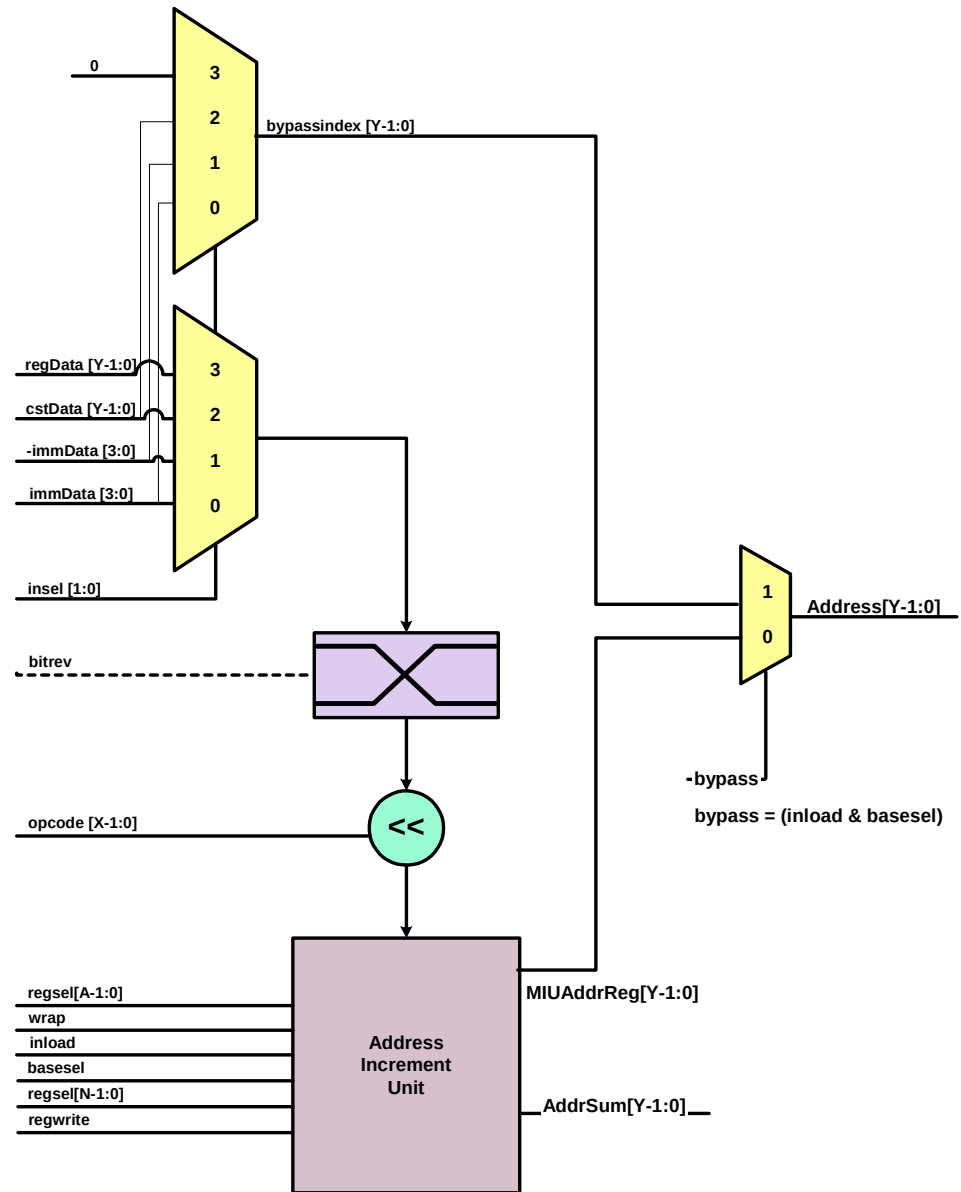


Figure 3-7. MIU Address Generation Unit

Figure 3-8 illustrates the function of the MIU address increment unit.

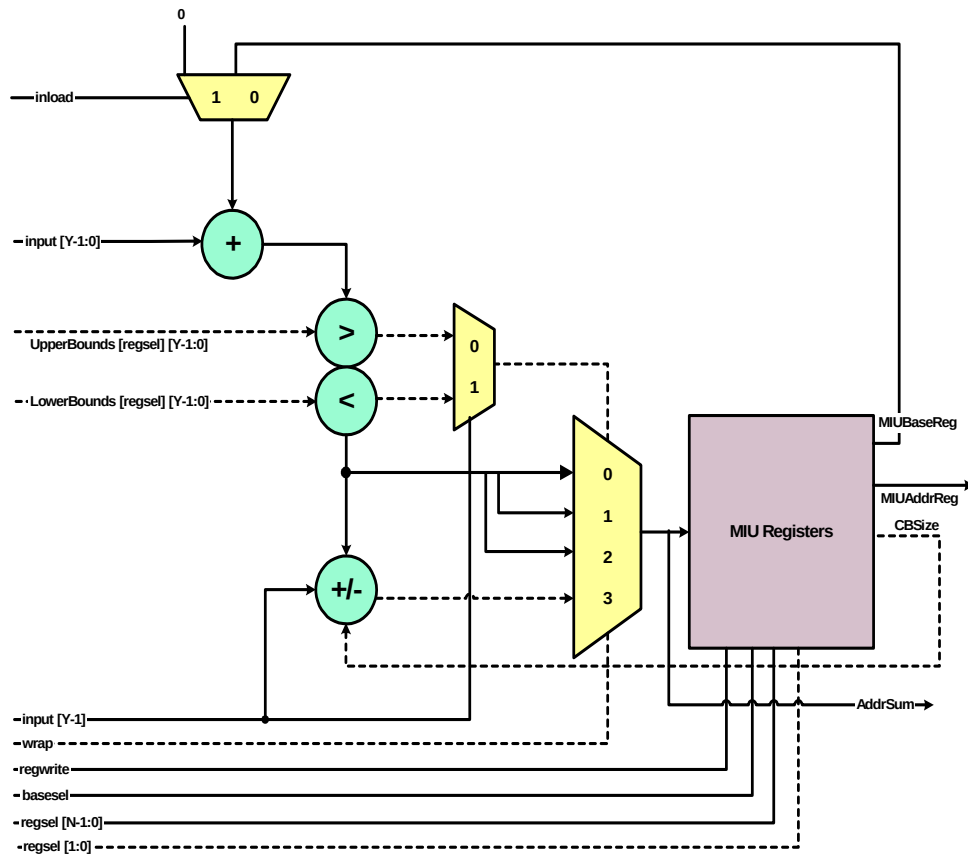


Figure 3-8. MIU Address Increment Unit

Control Unit

Instruction control is handled by the control unit. The following components are included in this unit:

- program counter,
- stack,
- branch address register,
- counter registers,
- condition register,
- repeat register,
- task parameter registers,
- control register,
- status register,
- interrupt mask register,
- program register,
- constant registers.

Table 3-6 lists the MOPs executed by the control unit and modified by the control unit registers.

Table 3-6. Control Unit MOPs

MOP	Description	Register(s) Affected
BR	Positive Conditional Branch	• program counter
CALL	Unconditional Call	• program counter • stack
CRI	Constant Register Initialize	• constant registers
CTD	Counter Decrement	• counter registers • condition register
CTS	Counter Set	• counter registers • condition register
SINTM	Set Interrupt Mask: • 1 = Disable Interrupts • 0 = Enable Interrupts	• interrupt mask register
ET	Positive Conditional End of Task	• program counter • task parameter registers • constant registers
INTR	Unconditional Interrupt	• program counter • stack • interrupt mask register
NBR	Negative Conditional Branch	• program counter
POP	Pop Out of Stack	• stack
PR_INI	Program Register Initialize	• program register
PR_ROT	Program Register Rotate	• program register
PR_WR	Program Register Write	• program register
PUSH	Push Into Stack	• stack
RET	Unconditional Return	• program counter • stack
RETE	Unconditional Return Enable	• program counter • stack • interrupt mask register
RPT	Repeat Next Instruction	• repeat register
SBR	Set Branch Address Register	• branch address register
SCR	Set Condition Register	• condition register
SCTL	Set Control Register	• control register
UBR	Unconditional Branch	• program counter
UET	Unconditional End of Task	• program counter • task parameter registers • constant registers

Task Queue Unit

Task control is controlled by the task queue unit. Jazz processors and other units within a PSA communicate using the task queue interface. The standard, high-priority, and interrupt task queues necessary for this communication are provided by the unit.

Table 3-7 defines the MOPs that are both executed by the Task Queue Unit and modify the various task queues with the PSA.

Table 3-7. Task Queue MOPs

MOP	Description	Queue(s) affected
HTQ	Queue High-Priority Task	High-Priority Queue(s)
ITQ	Queue Interrupt Task	Interrupt Queue(s)
TQ	Queue Standard Task	Standard Queue(s)

Computational Units

The Computational Units in the Jazz processor consist of functions generated and included in the CUs developed by Improv Systems Inc. The CUs are required to facilitate the following functions:

- perform data path operations in a single cycle;
- supply execute registers between the control input and the data path operator;
- supply result registers between the data path operator and module outputs;
- perform a No Operation (NoP) instruction when the control is invalid and the disable input is asserted;
- provide a base set of opcodes for initialization or pass through (the COPY MOP);
- drive all outputs at all times, even when they are never used;
- drive zeros on all outputs during the execution phase of an invalid control field.

CUs can be included from a library, or can be DDCUs that facilitate arithmetic and logic functions.

Data computation is handled by various CUs. Constant operands are supplied by the control unit, while global data operands are supplied by the data communications unit. The CUs operate on the source data, computing and storing the results within the global data result registers of the unit. The number and type of CUs within a Jazz processor can be configured. The designer may select from the base set of units (ALU, MAC and Shift), or may include a custom DDCU. The MOPs executed by DDCUs are not described in this document.

Table 3-8 lists the MOPs executed by the ALU, MAC and Shift units.

Table 3-8. Computational Unit MOPs

MOP	Description	Unit
ACA	Accumulator Accumulate	MAC
ACZ	Accumulator Zero	MAC
ADD	Addition	ALU
ADD16	Addition (16-Bit)	ALU
ADD5	Addition (5-Bit)	ALU
ADD8	Addition (8-bit)	ALU
AND	AND (bitwise)	ALU
ASR	Arithmetic Shift Right	Shift
CMP	Compare	ALU
CMP16	Compare (16-Bit)	ALU
CMP8	Compare (8-Bit)	ALU
CND	Condition (al0 only)	ALU
EQ	Relational Equal	ALU
GT	Relational Greater Than	ALU
GTE	Relational Greater Than or Equal	ALU
JEXP	Exponent Extraction	ALU
LSL	Logical Shift Left	Shift
LSR	Logical Shift Right	Shift
LT	Relational Less Than	ALU
LTE	Relational Less Than or Equal	ALU
MASK	Mask	ALU
MAX	Maximum	ALU
MIN	Minimum	ALU
MPS	Multiply Signed	MAC
MPU	Multiply Unsigned	MAC
MPW	Multiply Written (Copy)	MAC
NEQ	Relational Not Equal	ALU
OR	OR (bitwise)	ALU
SEL	Select	Shift
TRUE	Relational Always	ALU
UGT	Relational Unsigned Greater Than	ALU
UGTE	Relational Unsigned Greater Than or Equal	ALU
XOR	XOR (bitwise)	ALU

Figure 3-9 illustrates an overview of the CU architecture.

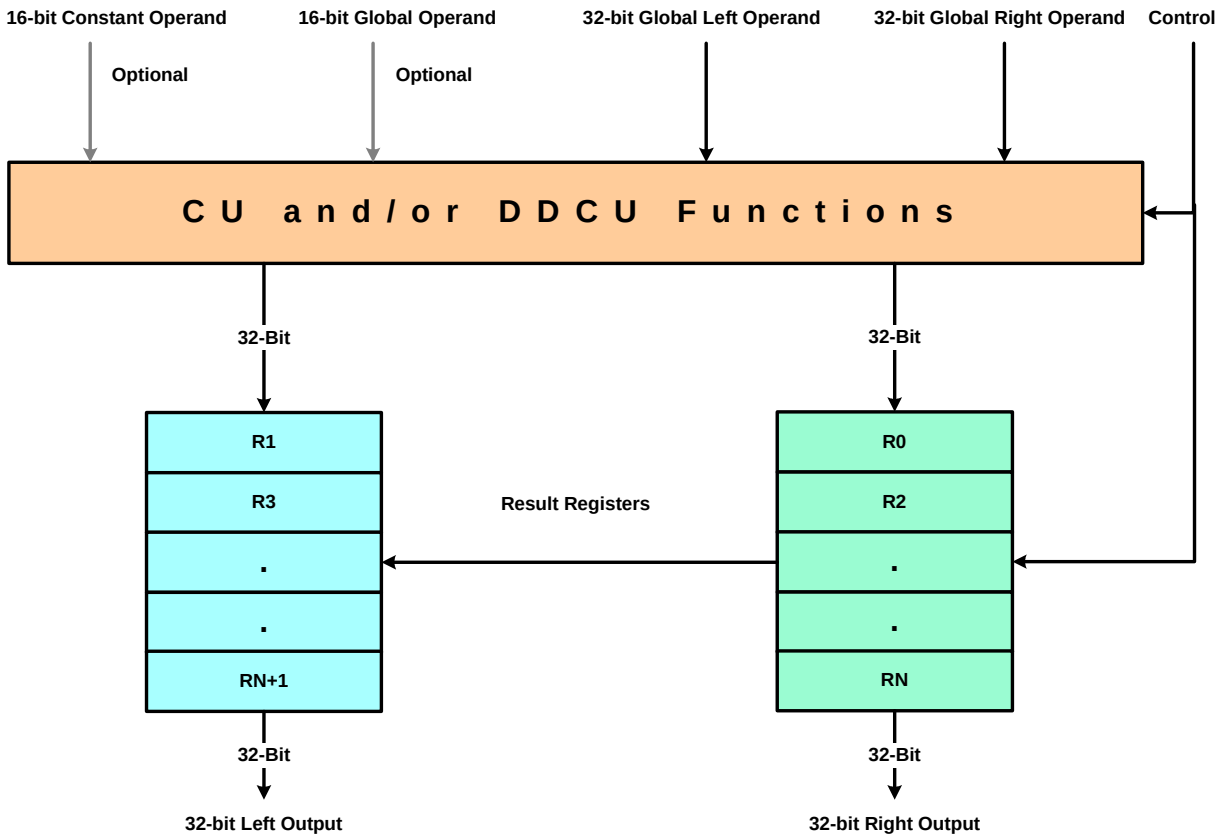


Figure 3-9. Computational Unit Architecture

Arithmetic Logic Unit

The Arithmetic Logic Unit (ALU) facilitates all of the arithmetic and logical operations of the Jazz processor, including the following:

- AND,
- OR,
- XOR,
- CMP,
- MASK,
- ADD,
- JEXP.

The Jazz processor consists of the following two types of ALUs:

- ALU Condition (this mandatory unit contains the condition register);
- ALU (this is an optional unit).

Some ALU operations can be performed on a sub-word basis. Saturation is also provided on a byte-wide basis.

Figure 3-10 illustrates an overview of the ALU architecture.

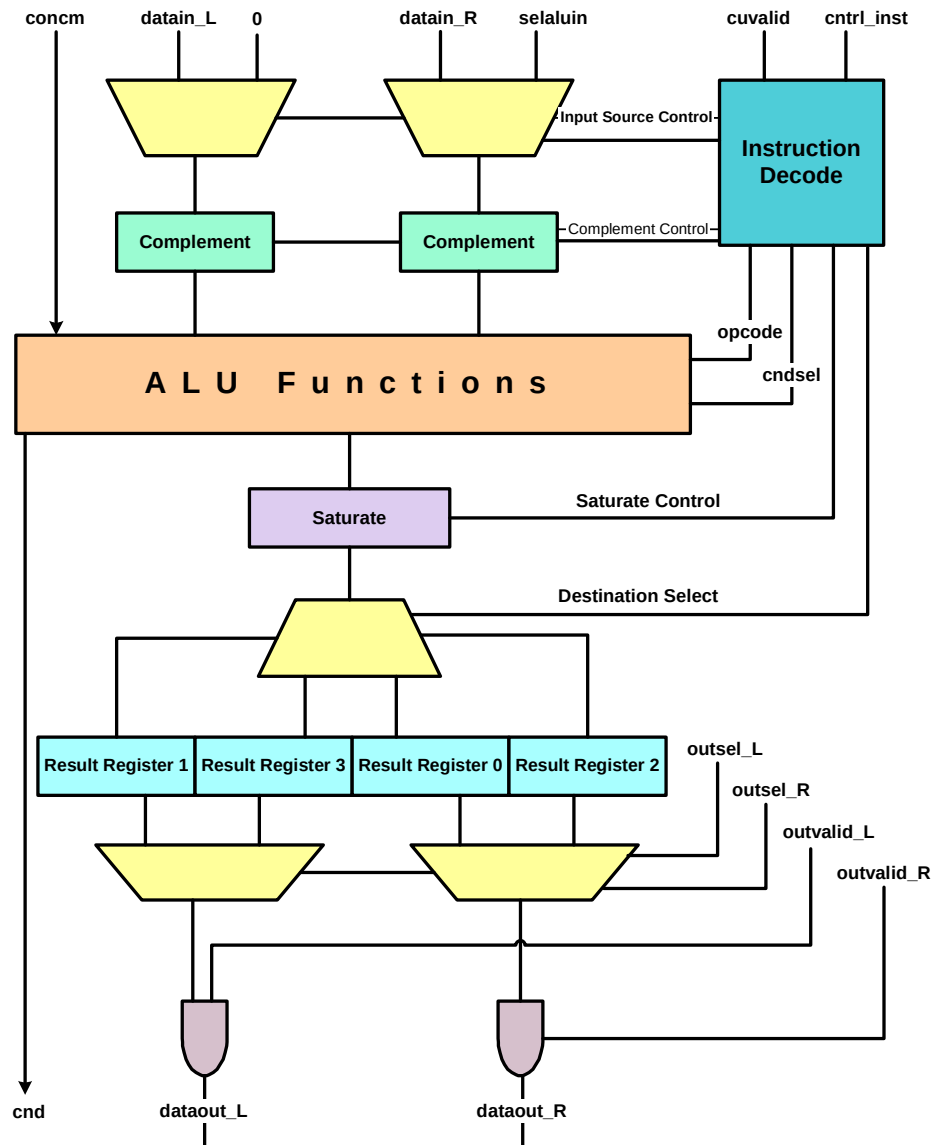


Figure 3-10. Arithmetic Logic Unit Architecture

Multiply and Accumulate Unit

The Multiply and Accumulate (MAC) Unit performs all multiply and accumulate functions using the output of multiply operations as input to accumulate operations.

The MAC unit operates as though the multiply and accumulate units are separate entities. The following characteristics of the MAC facilitate these type of operations:

- separate opcodes,
- separate result registers.

The result registers in the MAC are wide enough to handle the entire results. Multiply opcodes consist of signed, unsigned, and initialize (INIT). Accumulate opcodes consist of accumulate and clear.

Figure 3-11 illustrates an overview of the MAC architecture.

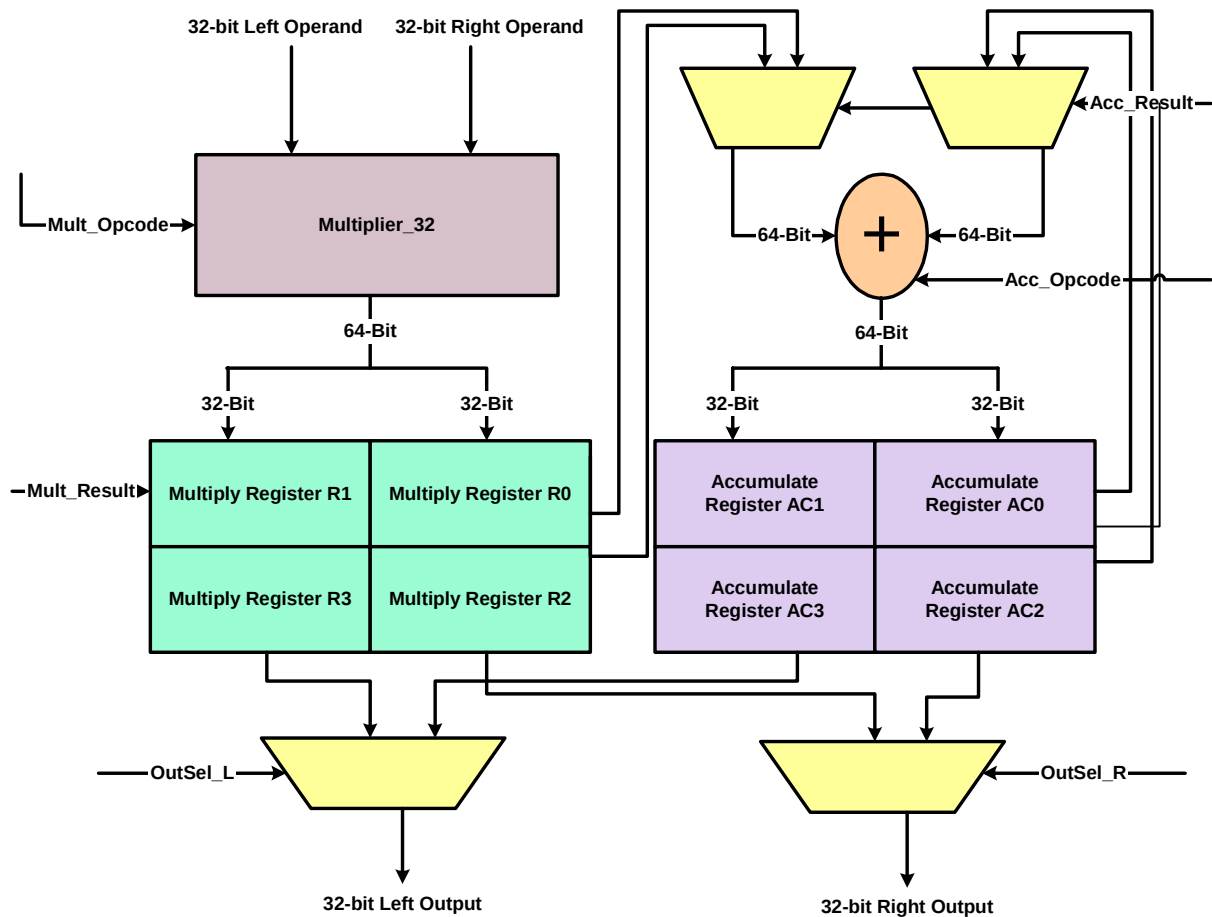


Figure 3-11. Multiply and Accumulate Unit Architecture

Shift Unit

The Shift unit shifts the entire data path width. The result registers are wide enough to hold the entire results (no truncation).

The following three types of shifters are available:

- Logical Shift Left (LSL),
- Logical Shift Right (LSR),
- Arithmetic Shift Right (ASR).

The features of rounding are defined below.

- Rounding is provided on a MOP-by-MOP basis.
- Improv Systems Inc. supports three types of rounding (RNE, RPI, RAZ).
- Rounding only applies to shift right operations.

There are many types of saturation available (32, 16, 12, 11, 10, 9, 8; both signed and unsigned).

Figure 3-12 illustrates an overview of the Shift architecture.

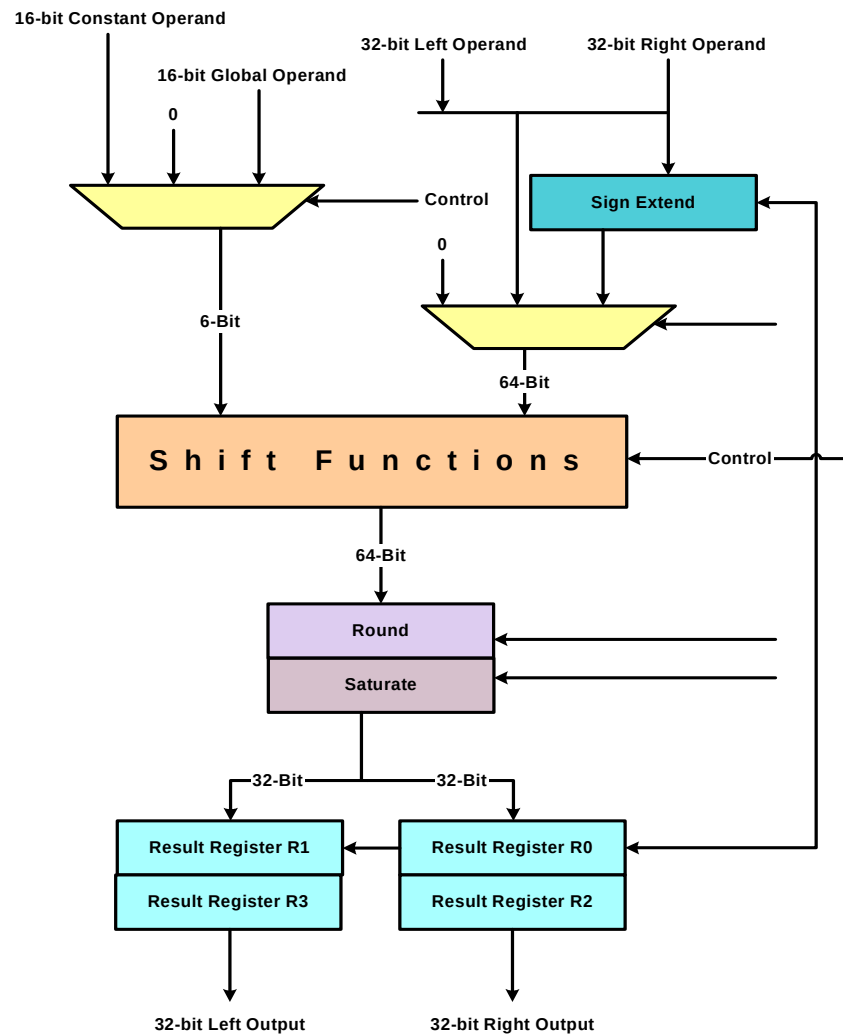


Figure 3-12. Shift Unit Architecture

Memory Interface Units

Data memory communication and address manipulation are controlled by various MIUs. Data is transferred between memories and the data communication unit. The data is communicated to and from these units using the global read result and address registers in the MIUs.

Table 3-9 lists the MOPs executed by MIUs and modified by MIU registers.

Table 3-9. Memory Interface Unit MOPs

MOP	Description	Memory & Register(s) Affected
LSS	Load Sum Sign Extended	• Read Register(s)
LSU	Load Sum Unsigned	• Read Register(s)
LWD	Load Write Data	• Read Register(s)
R16S	Read 16-Bit Sign Extended	• Read Register(s), • Address Registers
R16U	Read 16-Bit Unsigned	• Read Register(s), • Address Registers
R32	Read 32-Bit	• Read Register(s), • Address Registers
R8S	Read 8-Bit Sign Extended	• Read Register(s), • Address Registers
R8U	Read 8-Bit Unsigned	• Read Register(s), • Address Registers
SETA	Set Address Register (using primary map slot)	• Address Registers
SWA	Set Address Register (using secondary map slot [double-pumped read/write MIU])	• Address Registers
W16	Write 16-Bit Value to Memory	• Memory Register, • Address Register
W32	Write 32-Bit Value to Memory	• Memory Register, • Address Register
W8	Write 8-Bit Value to Memory	• Memory Register, • Address Register

MIUs require MOPs in a VLIW. MIUs provide the necessary address, data, and control signals to the memory. The hardware architecture supports an unlimited number of MIUs that can be configured for the following types of DSP addressing modes:

- bit reverse,
- circular buffer,
- indexed addressing.

The number of address registers can be configured. Support exists for either read/modify/write or read/byte/write sub-word operations.

The following MIUs are available for use with the Jazz processor:

- Single-Pumped (provides one READ or WRITE operation for each cycle);
- Double-Pumped (provides the following two operations for each cycle):
 - READ/ NOP (Read Only),
 - READ/ WRITE.

MIUs can include wait state support through a Ready signal selected by the user. MIUs can also include bus arbitration support through a Request/Grant/Acknowledge operation selected by the user.

System Control

The Q-Bus is a shared resource among Jazz processors and I-Blocks that are included in a PSA. The Q-Bus enables processors to communicate with each other by firing tasks. Each device on the Q-Bus contains a unique ID number, or processor address.

Figure 3-13 illustrates a sample Q-Bus.

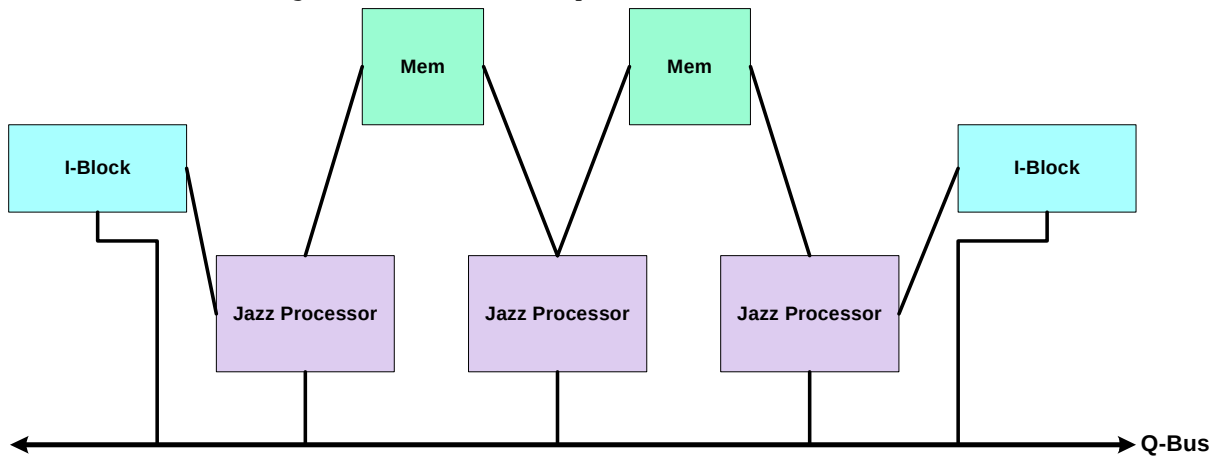


Figure 3-13. Sample Q-Bus

Hardware Stack

The depth of the 32-bit wide hardware stack is configured for either 0, 4, 8, or 16 words. Global result registers can access the stack through the PUSH and POP MOPs. The program counter can also access the stack through the CALL, INTR, RET and RETE MOPs.



The stack cannot be configured for 0 words when a configuration includes interrupts.

The PUSH MOP pushes the result register operand onto the top of the stack (TOS). The previous contents of each level of the stack are pushed into the stack, resulting in the loss of the lowest-level contents in the stack. When the *stack depth* number of PUSH MOPs occur without a POP MOP, a *stack overflow* occurs and data is lost from the stack.

The POP MOP forces the previous contents of each level of the stack to pop up into the stack, resulting in the loss of the top of the stack (TOS) contents. The bottom of the stack remains unchanged; therefore, the *stack depth* number of successive POP MOPs results in the entire stack being filled with the contents of the bottom of the stack. Typically, a COPY MOP that copies from the TOS is executed concurrently with the popping of the stack.

The CALL and INTR MOPs affect the hardware stack in a manner similar to the PUSH MOP. For the CALL MOP, the operand that is pushed onto the TOS is the program counter + 1. For the INTR MOP, the operand that is pushed onto the TOS is the program counter.

The RET and RETE MOPs affect the hardware stack in a manner similar to the POP MOP. These MOPs copy the contents of the TOS into the program counter concurrent with the popping of the stack.

Processor Configuration

Each processor can be configured to contain the following three different types of Task Queue FIFOs:

- Standard,
- Priority,
- Interrupt.

A standard task queue is required, and the size can be configured. The standard task queue is not a preemptive queue mechanism (i.e., no task in the queue starts before a prior task is complete). Upon completion of the current task, the processor executes the next task in the standard task queue. This process continues until the task queue FIFO is empty.

A priority task queue is optional, and the size can be configured. The priority task queue is not a preemptive queue mechanism. Upon completion of the current task, the processor examines the contents of both the standard and priority task queues. When there is a task on both queues, the processor first executes the task on the priority task queue. This continues until the priority task queue is empty.

An interrupt task queue is optional, and the size can be configured. The interrupt task queue is a preemptive queue mechanism (i.e., the current task being executed is interrupted so that the task in the interrupt queue can be executed). In cases where an interrupt task has been executed, the processor pushes the PC address to the stack.

The following components of the task queue FIFO can be configured:

- the size of each task queue FIFO;
- the depth of each of the FIFOs can be configured to hold more tasks;
- The width of the task queue FIFOs can be configured (based on Q-Bus variations).

The Task Type field cannot be configured.

The Q-Bus contains the following fields:

msb		lsb	
Task Type	Processor ID	Task Instruction Address	Parameter Data

Bit Width	Description
2	Task Type • 00 - No Task • 01 - Standard Task • 10 - Priority Task • 11 - Interrupt Task

Operation

The Q-Bus is a shared resource; therefore, units must arbitrate for control of the Q-Bus. This is accomplished in the cycle prior to use. Arbitration occurs during the execute stage of the queue instruction. The arbitration is a simple serial arbitration scheme, where the priority is based upon the connection of the processors in the arbitration chain. The Q-Bus is not expected to be an overloaded resource, because the average task length should be many cycles (more cycles than the number of processors).

Figure 3-14 defines the arbitration phases of the Q-Bus inside a processor. The relationship between the arbitration phases and the driving (or passing) of data is also illustrated.

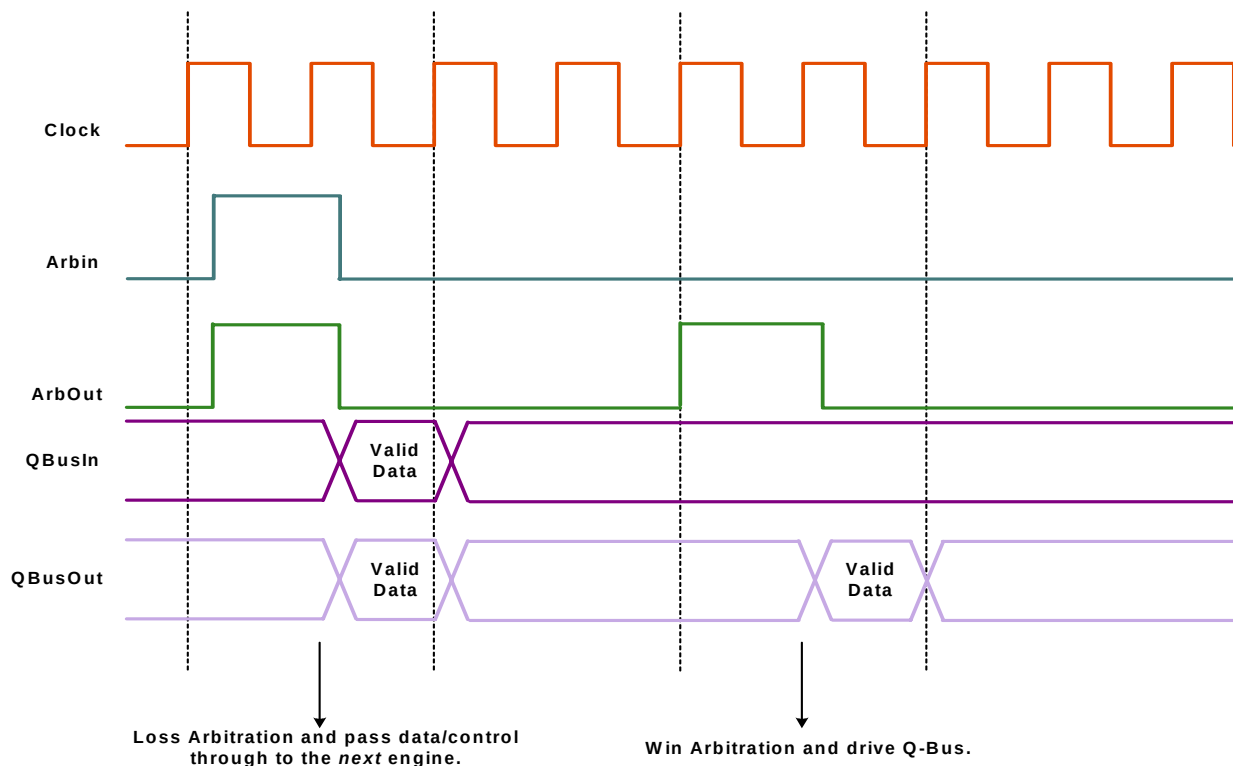


Figure 3-14. Q-Bus Arbitration Phases

Arbitration for the Q-Bus is accomplished in one clock cycle, with the data driven in the following clock cycle. When arbitration is won, the data driven on the Q-Bus is representative of the task fired from the specific processor. When arbitration is lost, the data driven onto the Q-Bus represents the data from a different processor (passing data through).

The Q-Bus is usually expected to be free; however, there is extra logic to minimize processor stalls due to Q-Bus resource contention. When a processor is executing a queue operation, but fails to win control of the Q-Bus, it stores the queue operation in a reservation station and proceeds to execute the following code. This process is possible because there are no dependencies within the task on the task queue operation. The processor only stalls if there is a queue operation in the reservation station and another queue operation is executed. When this occurs, the processor completes execution of the instruction. The task queue operation is stored in a second reservation station. The processor stalls until Q-Bus arbitration is won, since there are no remaining reservation stations. The processor restarts on the cycle during which Q-Bus arbitration is won.

When the Write signal on the Q-Bus is active, the task queue unit of the task processor places the data (representing a task) from the Q-Bus onto the bottom of the specified queue. The task processor is identified by the address that matches the one on the Q-Bus. A task processor address on the Q-Bus that equals 0 is intended as a *broadcast* task. All processors place this task in the correct queue.

When there is an end of task instruction, the instruction address and parameter data of a new task are read from the top of the task queue. When there is nothing on the queue, the Jazz processor stalls while waiting for a new task to queue. In this situation, the task processor tries to load a new task until it succeeds.

When it is read off the queue, the Q-Bus task address is input directly to the program counter in the instruction memory unit for loading at the next clock edge. The parameter data is stored in a register within the task queue block.

Attempts to write to an already full queue quietly fail; therefore, the task attempting to be queued is lost and is not executed.

Each Jazz processor decodes the processor ID on the Q-Bus and determines whether or not to write that Q-Bus information into the FIFO.

Implementation

To avoid difficulties with testing, the Q-Bus is implemented as a pair of buses, as opposed to a tri-state bus implementation. One bus goes from device to device, working through each one. The other bus goes from device to device in the opposite direction. Only the processor that has control of the Q-Bus (won arbitration) drives out 1s in the appropriate position on the Q-Bus.

All other processors drive 0s for all positions and/or the signal, with the signal coming in from the adjacent processor. All processors to the right of the driving processor get the driven data on one bus (and the other bus is all 0s). The processors to the left see the driven data on the opposite bus (and the other bus is all 0s). Each processor locally ORs the data from both of these buses to get the final Q-Bus data. Any one processor can communicate to all processors on a shared pair of buses without tri-state drivers.

Instruction Pipeline

The following instruction pipeline examples are described in this section:

- Branch Instruction,
- Call Immediate,

- Repeat.

Figure 3-15 illustrates the Branch instruction executing in the pipeline. This process is described below.

1. A Branch instruction triggers a read from the Branch Register in *Cycle 4*.
2. The read updates the program counter with the branch address.
3. The delay slot of the branch instruction executes in *Cycle 5* while the pipeline fills with the first instruction from the branch.

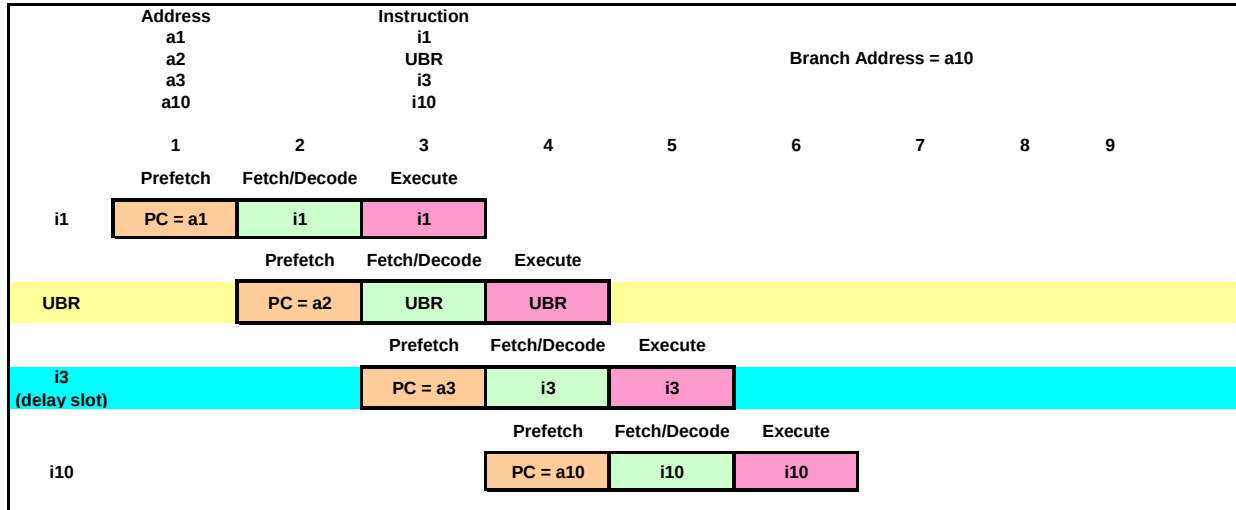


Figure 3-15. Branch Instruction in the Pipeline

Figure 3-16 illustrates the Call Immediate instruction executing in the pipeline. This process is described below.

1. A Call instruction triggers a branch to the Immediate address in *Cycle 4*.
2. The branch updates the program counter with the immediate address. The RTN address (PC + 1) is also pushed to the top of the stack.
3. The delay slot of the Call instruction executes in *Cycle 5* while the pipeline fills with the first instruction from the branch.

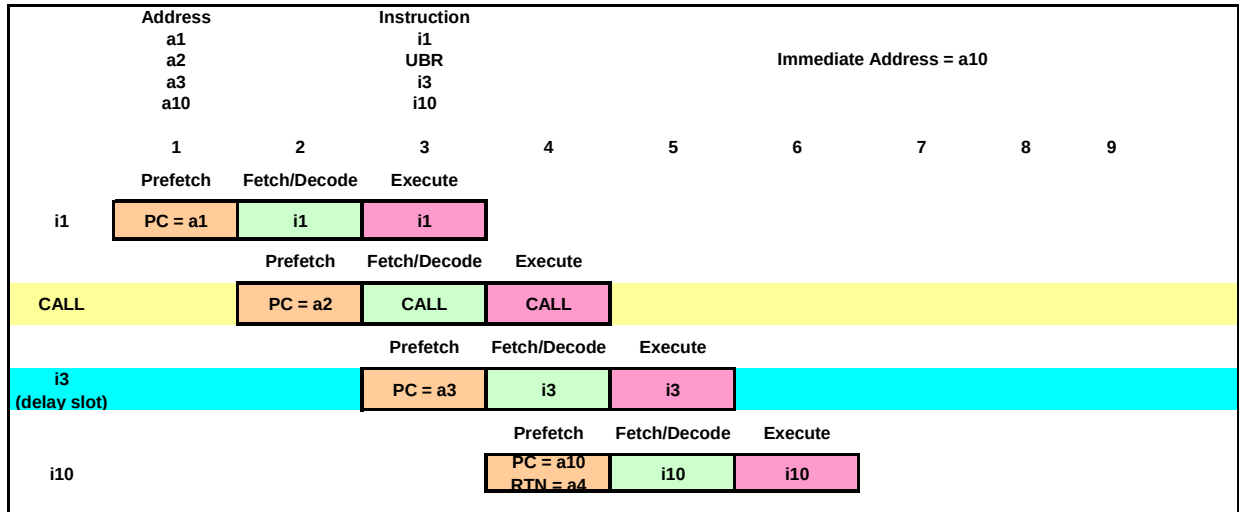


Figure 3-16. Call Immediate Instruction in the Pipeline

Figure 3-17 illustrates the Repeat instruction executing in the pipeline. This process is described below.

1. Instruction (i2) is a RPT 1 instruction (repeat next instruction one time). When the RPT instruction is executed in *Cycle 4*, the Program Counter pre-fetches the same instruction that was pre-fetched in *Cycle 3* (i3).
2. The NPC register is updated with the program count that would have occurred if there was no Pre-fetch instruction. The repeat was only active for one instruction; therefore, the repeat active flag is negated in *Cycle 5* and the program counter is updated with the contents of the NPC register.
3. Instruction (i3) is executed in both *Cycles 5* and *6*. Repeated instructions cannot be interrupted; however they still respond to Waits.

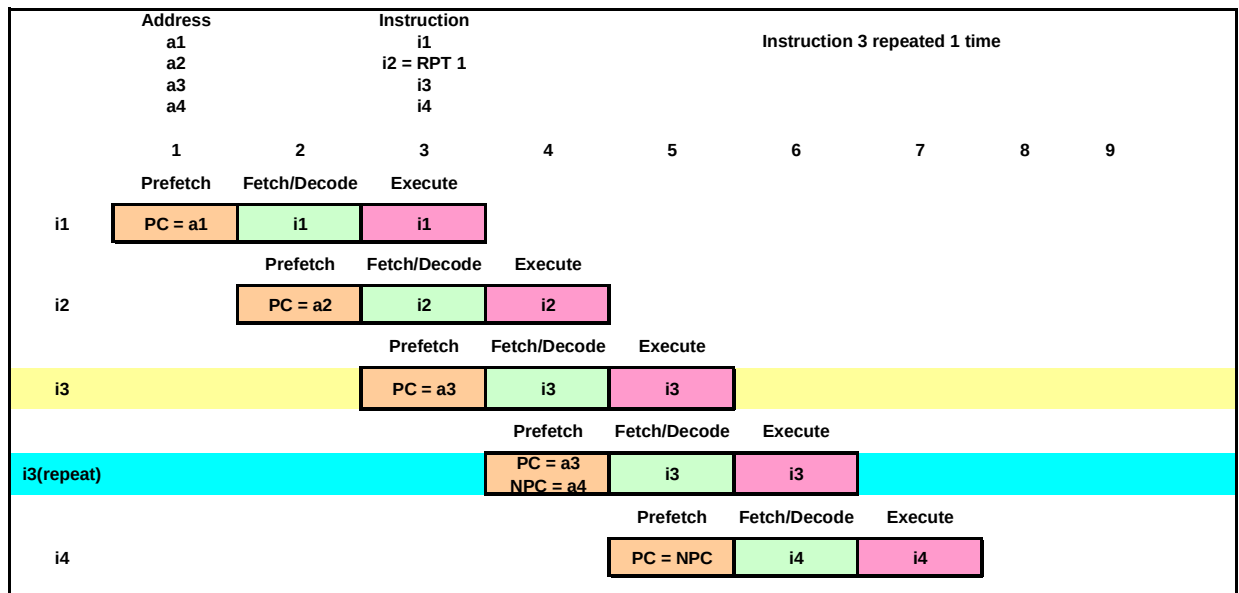


Figure 3-17. Repeat Instruction in the Pipeline

Interrupts

Jazz processors operate on the premise that tasks are not preemptive (when a task is fired, it cannot be stopped). While this suits the majority of existing applications, there remain are a number of instances where this operation is not required. To accommodate these few instances, Improv Systems Inc., has implemented a separate queue (and associated queuing logic) that interrupts a task (referred to as an interrupt queue).

Interrupt Queue

A designer can compose a processor with or without the interrupt task queue. From an operations standpoint, the interrupt task queue is designed similar to other task queues (standard and priority). An interrupt task (denoted by the Task Type field) that matches the Engine ID is placed in the interrupt queue (when the Enable Interrupt Task Queue bit in the control register is set). Where an interrupt task is present in the interrupt task queue (and the interrupt mask bit is turned off) when the current executing instruction is complete, the interrupt task is read from the task queue (e.g., the interrupt task is fired).

Figure 3-18 illustrates a block diagram of the interrupt task queue functionality.

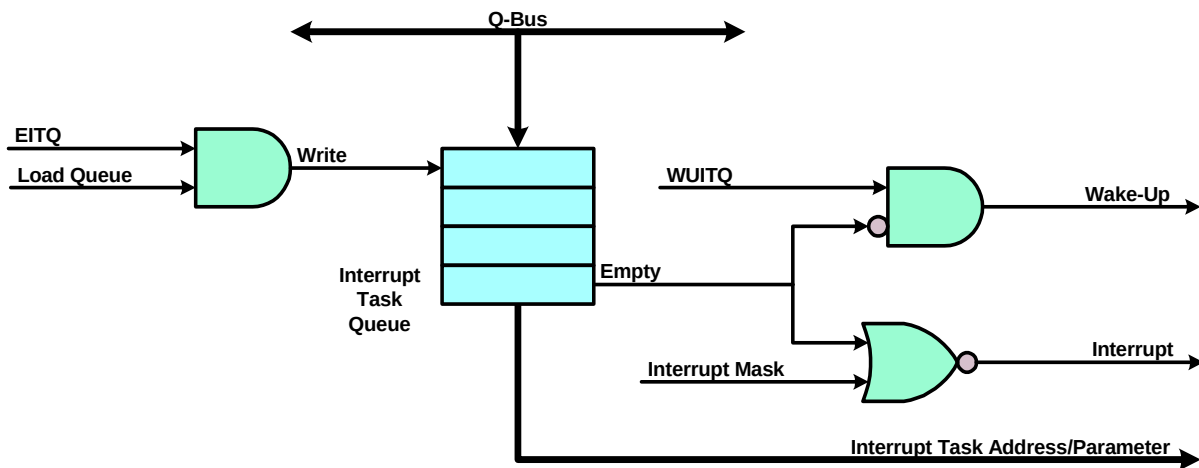


Figure 3-18. Interrupt Task Queue

When the interrupt task is fired, the current value of the program counter (PC) is pushed to the Top of the Stack (TOS) and replaced with the immediate address from the interrupt queue. The interrupt instruction (INTR) executes with a NOP inserted in the following cycle, and the PC is updated with the interrupt queue address. When the INTR instruction executes, the Interrupt Mask bit is automatically set to disable further interrupts from the interrupt task queue. To enable *nested* interrupts, this bit must be de-asserted. The Interrupt Service Routine (ISR) then runs. The second to last instruction of this routine is a Return and Enable Interrupts (RETE), with the last one being the delay slot. The result of this instruction is the updating of the PC with the TOS. This enables the instruction run after the delay slot of the RETE instruction to be the one that was present when the initial INTR was inserted.

Figure 3-19 illustrates the pipeline response of the interrupt. The ISR in this case is trivial because the only instruction is the RETE. The diagram illustrates the RETE instruction as *ISR0*, and the delay slot as *ISR1*.

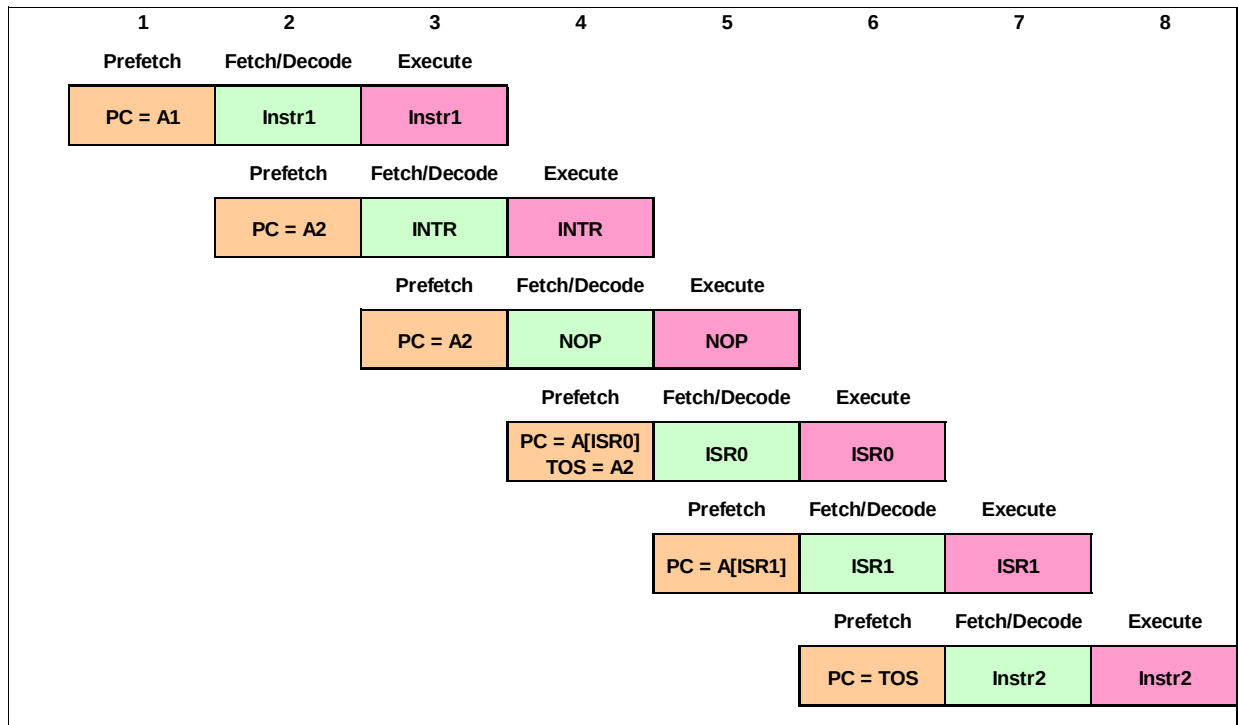


Figure 3-19. Interrupt Pipeline Response

Control

Enable Interrupt Task Queue (EITQ) is a bit that, when set, enables interrupt tasks (denoted by the task type on the Q-Bus) to be written into the interrupt task queue. When a processor is composed with an interrupt task queue, this bit is reset to 1 to automatically enable this feature. The bit is set to 0 to disable it.

Wake-Up to Interrupt Task Queue (WUITQ) is a bit that indicates whether or not an interrupt task *wakes* a processor up from sleep mode. This bit is set to 1 to enable this feature.

The Interrupt Mask is a bit that, when set, masks the existence of a task in the task queue. The existence of such a task does not cause an interrupt to occur. This bit is automatically set by the processor when an interrupt task is fired (to prevent nested interrupts). The ISR can then enable or disable interrupts with the EINT and DINT MOPs (respectively).

Caveats

Some instructions cannot be interrupted. For example, the INTR instructions are not inserted into the Fetch cycle when a Branch instruction is executing, the processor is waiting (or stalled), a repeated instruction is executing, or while the fetch instruction is decompressing.



Chapter 4 Assembly Language

This chapter describes the assembly language fields within the Jazz processor. The fields are listed alphabetically, and correspond to the fields defined for each MOP described in *Chapter 5, Instruction Set Summary*.

The Jazz processor contains a control unit and a set of Memory Interface Units (MIU) and Computational Units (CU). The control unit reads μ -task instructions and in turn controls the memory and computational units within the μ -task engine. These μ -task instructions are Very Long Instruction Words (VLIW) that support parallel execution of multiple resources within the engine.

The Jazz processor assembly language follows a number of conventions defined in *Table 4-1*.

In This Chapter:

- Syntax • lvi
- Memory Addressing Modes • lvi
- Assembly Language Fields • lix
- Addition & Subtraction • lxx



The example below illustrates a complete instruction generated by the assembler. Note the use of the conventions defined in *Table 4-1*.

```
@0x1 0xd
{
    757
    0:0:758 PR_INI 0x0 0xc28a95be 0x0 0x0 0x0 0x0 0x0 0x0 ;
}
@0x1 0xe
{
    760
    0:0:770 R8U smr.p.r0 smr.p.ar5 smr.p.ar5 0xadbc wrap
    0:0:764 W16 smr.p VoAcc0.r0 smr.p.ar3 smr.p.ar2 ctl.cst3
    0:0:768 R32 sml.x.r0 sml.x.ar0 sml.x.ar0 mp0.ac2
    0:0:767 W32 sml.x VoAcc0.r3 zero sml.x.ar2 zero
    0:0:765 R16S pdm.x.r0 pdm.x.ar7 pdm.x.ar7 ctl.cst6
    0:0:763 W16 pdm.x cnt.c2 pdm.x.ar6 pdm.x.ar6 al0.r0 wrap
    0:0:761 MPU mp0.r3:mp0.r2 sml.x.r0 sh0.r0
    0:0:766 LSL sh0.r1:sh0.r0 sign:smr.p.r0 mp0.r3 R U9
    0:0:769 userdef:FMLT VoAcc0M4FMLTRIA2II VoAcc0.r1 mp0.ac2 VoAcc0.r3 zero
    zero
    0:0:762 ADD al0.r1 zero al0.r1 F F CL NS ctl.cst1 ;
}
```

Syntax

The Jazz processor assembly language syntax is defined in *Table 4-1*. The table lists the symbol and a definition of the symbol.

Table 4-1. Assembly Language Syntax

Symbol	Definition
{	Open curly brackets denote the beginning of a Very Long Instruction Word (VLIW) comprising a set of complete instructions. The number immediately following the open curly bracket denotes the line number in the code.
}	Closed curly brackets denote the end of a VLIW.
;	A semi-colon denotes the end of a complete instruction. A single instruction can be composed of several Micro Operations (MOPs).
All CAPS	Capital letters near the beginning of an operand are the mnemonic. The mnemonic corresponds to the specific micro operation selected for the instruction.

Memory Addressing Modes

The Jazz processor includes the following eight types of addressing modes:

- direct,
- indirect,
- auto-increment,
- displacement,
- indexed,
- circular buffer,
- bit-reversed.

Each data MIU can perform the following two types of addressing operations:

- pipelined displacement
- indexed addressing.

Table 4-2. lists the memory units referenced by the Jazz processor and the corresponding dimensions.

Table 4-2. Memory Units

Unit Name	Mnemonic	Width	Depth (max)
Instruction Memory	*	~128	64K
Left Shared Data Memory	SML	32	64K
Private Data Memory	PDM	32	64K
Right Shared Memory	SMR	32	64K

Direct Addressing Mode

Direct or absolute addressing forms the data memory address directly from the instruction stream. This operation is accomplished by generating the address operand from one of the following:

- the 16-bit immediate value within the VLIW,
- a 16-bit constant register within the control unit.

Constant registers are loaded by 16-bit immediate values from the instruction stream.

The following is an example of the direct addressing mode:

```
{ WR pdm.x, mp0.ac2, 148; }
```

Indirect Addressing Mode

Indirect or register-deferred addressing forms the data memory address from the pipelined computed result. This is accomplished by generating the address operand from any of the processor result registers, and pipelining it using an address register. The least significant 16 bits of the result registers are used in address operand generation.

The following is an example of the indirect addressing mode:

```
{ WR smr.p, mp0.ac0, smr.p.ar1; }
```

Auto-Increment Addressing Mode

Auto-increment addressing forms the data memory address from an incremented result. This operation is accomplished by generating the address operand from any MIU address register. The MIU add operator is then used to post-increment the address register by a constant value. The constant value may be any one of the following:

- a 16-bit immediate,
- a 16-bit constant register,
- a value of 1, 2, or 4 generated by an MIU operation.

Depending upon the type of Load or Store operation (8-bit, 16-bit, or 32-bit), the MIU can generate an incremented value (1, 2, or 4 respectively).

The following is an example of the auto-increment addressing mode:

```
{ WR sml.x, al0.r0, sml.x.ar3, sml.x.ar3, 0x4; }
```

Displacement Addressing Mode

Displacement addressing forms the data memory address from the sum of a computed result and a displacement offset. This is accomplished by generating the computed operand from any of the processor result registers, and the offset operand from either the 16-bit immediate or 16-bit constant registers. The sum of the two operands is performed by the add operator in the MIU. An address register is used as a pipeline register to store the result. The pipeline enables the higher operating frequency of the *double-clocked* MIUs.

The following is an example of the displacement addressing mode:

```
{ WR sml.x, sh0.r0, sml.x.ar3, sml.x.ar2, 0x0084; }
```

Indexed Addressing Mode

Indexed addressing forms the data memory address from a computed index. This is accomplished by generating both computed values within any of the processor result registers. The sum of the two operands is performed by the add operator in the MIU. An address register is used as a pipeline register to store the result. The pipeline enables the higher operating frequency of the *double-clocked* MIUs.

The following is an example of the indexed addressing mode:

```
{ WR sml.x, sh0.r0, sml.x.ar3, sml.x.ar2, al0.r0; }
```

Circular Buffer Addressing Mode

The circular buffer operator can be configured. If it is configured, it may be applied to an address register sum. The circular buffer operator wraps the address sum at the lower and upper address limits of the buffer. The circular buffer operator option is compatible with all Jazz processor (Version 2.2) addressing modes (with the exception of direct addressing).

The following is an example of the circular buffer addressing mode:

```
{ SETA smr.p, zero, smr.p.ub0, zero, ctl.cst2;
  SETWA smr.p, zero, smr.p.lb0, zero, al0.r3; }
{ WR smr.p, al0.r1, smr.p.ar1, smr.p.ar0, mp0.ac2, wrap; }
```

Bit-Reverse Addressing Mode

The bit-reverse operator can be configured. If it is configured, it may be applied to an address register increment value, and mirrors a portion of that increment value. The bit-reverse operator option is compatible with all Jazz processor (Version 2.2) addressing modes (with the exception of direct addressing).

The following is an example of the bit-reverse addressing mode:

```
{ SETA smr.p, zero, smr.p.nb0, zero, ctl.cst2; }  
{ WR smr.p, al0.r1, smr.p.ar1, smr.p.ar0, mp0.ac2, bitrev; }
```

Assembly Language Fields

The Jazz processor assembly language fields are listed alphabetically on the pages that follow. Refer to *Chapter 5, Instruction Set Summary* for the values applicable to each field within individual MOPs.

Amount

In a Shift unit operation, the Amount field specifies the number of bits the data is shifted. The least significant 6 bits (unsigned) of a register or constant are used. The shift amount must be in the range of 0 to 63. This field must contain one of the following values:

constant value | 32-bit register | zero

CarryIn

In an ALU operation, the CarryIn field specifies the input carry value for addition operations. Although al0 generates a carry out flag, each arithmetic unit has its own independent carry register. This field must contain one of the following values:

CL	Specifies a 1 for carry in if the right operand is complemented; otherwise, a 0 is assigned to this field.
CO	Specifies that the input carry value is the carry register (which is set to the carry out generated by the same arithmetic unit in the previous cycle.) If no arithmetic operation is performed in a given cycle, the carry register retains its value from the previous cycle.

If the complement operation or carry in selection is specified as dependent on the sign bit, the sign bits of upper and lower 8-bit values are independently processed. For example, in an ADD8 instruction, if the right input is complemented when it is negative, then the upper eight bits are complemented if bit 15 is *one*. The lower eight bits are complemented if bit seven is *one*. Similarly, if the carry in is specified as the sign bit of the right input (the previous example with the CarryIn field set to CL), the carry in for the addition of the upper eight bits is bit 15 of the left input and the carry in for the lower addition is bit seven.

Cmask

In an ALU operation, the Cmask field is used by the Condition Operation (CND), and is needed by the assembler to generate the syntax for other ALU_32 instructions. In ALU micro operations other than CND, the Cmask field is not used, but is filled with the value *ctl.cst1* to indicate that it is ignored. This field must contain one of the following values:

ctl.cst1 | constant value | zero

Six of the upper eight bits of the condition mask correspond to six conditions (one bit per condition). Six of the lower eight bits also correspond to six conditions (one bit per condition). The remaining unused bits are reserved for additional conditions in the future. A condition is selected by a value of 1 in the corresponding bit position. The upper bits select the positive conditions and the lower bits select the negative conditions.

In addition to the positive and negative condition specifiers, the Cmask field also contains the NEG control flag. When this flag is 0, the Set Condition Register (SCR) operation behaves exactly as described above. However, when the NEG control flag is 1, an SCR instruction causes the condition register to be set to the opposite value as described above. This means that when the NEG flag is set, SCR sets the condition register to FALSE if any selected positive conditions are TRUE, or if any selected negative conditions are FALSE. Otherwise the condition register is set to TRUE.

Datan

In a Constant Register unit operation, the Data Number (Datan) field specifies the values to be stored in the constant registers *cstn*, where *n* is a value from one (1) through fourteen (14). This field must contain the following value:

16-bit immediate value

There are 15 Constant Registers, ranging from Cst0 - Cst14. Constant Register 0 (Cst0) always contains the 16-bit immediate from the current standard instruction. Constant Registers 1-14 are initialized by the Constant Register Initialization (CRI) instruction. The CRI instruction updates as many complete 16-bit registers that the instruction word width can support. The CRI Index is used to ensure that all of the Constant Registers can be initialized for all instruction word widths.

Dest.a

In MIU read and write operations, the Destination Address (Dest.a) field specifies the destination address register. If the Dest.a field is not zero, the sum of the address and the increment value (Src.inc) is loaded into this address register. This field must contain one of the following values:

address register | zero

Dest.m

In an MIU write operation, the Destination Memory (Dest.m) field specifies the destination memory unit. It must be either private data memory, or one of the shared memory units connected to the task engine. This field must contain one of the following values:

pdm | sml | smr

Dest.r

In an MIU operation, the Destination Register (Dest.r) field specifies the destination read register. It must be one of the read registers in the private or shared memory units, or zero. This field must contain one of the following values:

mname.mrport.r0 | uname.rx | uname.ry | uname.rp | zero

Destination

In all operations, the Destination field specifies the destination register of the result. It must be one of the four 32-bit result registers within the arithmetic unit. In a MAC_32 operation, this field must contain one of the following values:

uname.ac1:uname.ac0 | uname.ac3:uname.ac2

In the Shift, ALU and ADLU operations, this field must contain one of the following values:

uname.r0 | uname.r1 | uname.r2 | uname.r3

Index

In a Constant Register unit operation, the value in the Index field corresponds to the start location of the initial Immediates when they are loaded into the constant register. This field must contain one of the following values:

0 | 1 | 2 | 3

The values for the Index field correspond to the following constant registers:

Index	Cst
0	1
1	5
2	9
3	13

L.comp

In an ALU operation, the Left Complement (L.comp) field specifies whether the left input is complemented or not complemented. This field must contain one of the following values:

F	The input is not complemented.
T	The input is complemented.

L.operand

In an ALU or ADLU operation, the Left Operand (L.operand) field specifies the left input data value. This field must contain one of the following values:

32-bit register | zero

Opcode

In all operations (excluding ALU operations), the value in the Opcode field corresponds to the mnemonic. The opcode for ALUs references both the mnemonic and the saturate fields. The opcodes for all operations are defined in *Chapter 5, Instruction Set Summary*.

Parameter

In a Task Queue operation, the Parameter field specifies a 16-bit parameter that is queued with the μ -task. The parameter value is loaded into the lowest number read address registers of the engine (pdm.rax0, sml.rax0, and smr.rap0) when the task is initiated. This field must contain one of the following values:

constant value | 32-bit register

R.comp

In an ALU operation, the Right Complement (R.comp) field specifies whether the right input is complemented, not complemented, complemented if it is negative, or complemented if it is positive. This field must contain one of the following values:

F	The input is not complemented.
N	The input is complemented if it is negative.
P	The input is complemented if it is positive.
T	The input is complemented.

R.operand

In ALU and ADLU operations, the Right Operand (R.operand) field specifies the right input data value. This field must contain one of the following values:

32-bit register | 5-bit immediate | 4-bit immediate

Rounding

In a Shift unit operation, the Rounding field specifies whether the result is rounded or not rounded. This field must contain one of the following values:

NR	No rounding is performed.
R	The shifted result is rounded.

When rounding is specified in conjunction with a right shift (LSR, ASR) of N bits (where N can vary from 0 to 31), the N least significant bits of the source are examined. If N is greater than 16, only the 16 most significant bits of the N bits are examined. These bits are used to perform rounding, using a round to the nearest algorithm. When the original value is near two rounded values, the even rounded value is chosen as the result. This algorithm works according to the following rules:

- If the most significant of the shifted out bits (bit $N-1$ of the source) is 0, the shifted result is *rounded down*.
- If bit $N-1$ is one, and at least one other of the N least significant bits is also one, the shift result is *rounded up*.
- If bit $N-1$ is one, and all other of the N least significant bits are 0, the shifted result is *rounded up* (for odd values) and *rounded down* (for even values).

Rounding down means the shifted out bits are truncated and lost. *Rounding up* causes a 1 to be added to the least significant bit of the shifted result (e.g., the shifted result is incremented). This algorithm works correctly for both positive and negative signed and unsigned values.

Saturate

The Saturate field specifies the kind of saturation (if any) that is performed. This field must contain one of the following values:

NS	Saturation is not performed.
S10	Saturation is performed to a 10-bit signed result.
S11	Saturation is performed to an 11-bit signed result.
S12	Saturation is performed to a 12-bit signed result.
S16	Saturation is performed to a 16-bit signed result.
S32	Saturation is performed to a 32-bit signed result.
S8	Saturation is performed to an 8-bit signed result.
S9	Saturation is performed to a 9-bit signed result.
U10	Saturation is performed to a 10-bit unsigned result.
U11	Saturation is performed to an 11-bit unsigned result.
U12	Saturation is performed to a 12-bit unsigned result.
U16	Saturation is performed to a 16-bit unsigned result.
U32	Saturation is performed to a 32-bit unsigned result.
U8	Saturation is performed to an 8-bit unsigned result.
U9	Saturation is performed to a 9-bit unsigned result.

Saturation is performed after rounding. When saturation is enabled, the upper 32 result bits (outside the saturation range) are undefined. For S16, S12, S11, S10, S9, and S8 the result is sign extended to 32 bits. For U16, U12, U11, U10, U9, and U8 the result is zero filled to 32 bits (the width of data communication is 32 bits). Refer to the section entitled *Saturation* for additional information about saturation in Shift_32 unit operations.

Source

In a Shift unit operation, the Source field specifies the input data to be shifted. Only a concatenated 32-bit global register can be specified. The type of saturation (signed/unsigned along with the width) specifies the format of the result, but does not imply anything about the format of the source data. This field must contain one of the following values:

32-bit register:32-bit register | 0:32-bit register | 0

In a Counter Register operation, the Source field specifies the input (set) data value. If this field contains a counter register, it must match the register in the destination field. This field must contain one of the following values:

constant value | 32-bit register | *uname.r0* | *uname.r1* | *uname.r2* | *uname.r3*

In a Branch Register operation, the Source field specifies the value for the branch address register. The lower 12 bits of a constant value or register are loaded into the branch address register. This field must contain one of the following values:

32-bit register | constant value

In a Condition Register operation, the Source field specifies the condition mask for assignment of the condition register. The values for the Source field correspond to the following condition mask symbols:

Bit #	Symbol	Description
0	!tc	Enables the NOT terminal count condition.
1	!ov	Enables the NOT overflow condition.
2	!co	Enables the NOT carry condition.
3	!ng	Enables the NOT negative condition.
4	!zr	Enables the NOT zero condition.
5	!cnd	Enables the NOT last condition.
8	tc	Enables the terminal count condition.
9	ov	Enables the overflow condition.
10	co	Enables the carry condition.
11	ng	Enables the negative condition.
12	zr	Enables the zero condition.
13	cnd	Enables the last condition.
15	pol	Negates the condition register input.

Src.a

In an MIU operation, the Source Address (Src.a) field specifies the source of the address value. It may be the lower 16 bits of a global 32-bit register, a constant value, or an address register associated with the read port. If local address registers are specified for the Dest.a and Src.a fields, they must both share the same register number. This field must contain one of the following values:

32-bit register | constant value | address register | zero

Src.dat

In an MIU operation, the Source Data (Src.dat) field specifies the source for the data written to memory. This field must contain one of the following values:

32-bit register | address register | zero

Src.inc

In an MIU operation, the Source Increment (Src.inc) field specifies the source of the increment value. This field may be the lower 16 bits when a global 32-bit register is selected. This field must contain one of the following values:

32-bit register | constant value | zero | 1 | 2 | 4

Task.addr

In a Task Queue operation, the Task Address (Task.addr) field specifies the address of the μ -task being queued. The upper four bits of the address specify the task engine, IO engine, or IO module. The lower 12 bits of the Task.addr field specify the local task address in instruction memory. This field must contain one of the following values:

constant value | 32-bit register

Constant Values

Constant values are 16-bit constants that are stored in a set of special purpose registers. The function of constant values is to enable access to constants by multiple μ -task operations within a μ -task instruction, while keeping the instruction width as compact as possible. There are seven general constant registers (cst1 - cst7). There are also seven address constant registers (cst8 - cst14). These 14 constant registers can only be set using a special full-width instruction that includes constant values as immediate values.

Various μ -task instructions can use the constant values as input operands. For an operation that allows the use of a constant as an operand, either a general constant register (cst1 - cst7) or a numeric value can be specified. For the address specification (Src.a) in both memory read and memory write port operations, and for the parameter (Parameter) in task queue operations, the address constant registers (cst8 - cst14) can also be specified. The address constant registers are in addition to the general constant registers or a numeric value.

Since there is only a single immediate field, only one unique numeric constant value can be used for all μ -OPs in an instruction. However, there are no limits to the number of constant registers that can be referenced in a μ -task instruction.

Registers

All units contain address or data registers. These registers are used to capture computed results and locally retain intermediate values. In each μ -task instruction, the registers of a unit may be updated with computed or input values. If a register is not explicitly modified in an instruction, it retains the previous value assigned to it (with the exception of the `ctl.pr` register).

Data values computed by a unit may be written only to a register within that unit. A destination register must always be specified for an operation. (It is not possible to perform an operation without modifying a result register for the purposes of updating condition flags.)

Registers supply the data inputs (operands) for memory and datapath units. Registers that may only be used (read) within a unit are referred to as local registers. Registers that may be used as inputs by any unit (referenced outside of the containing unit) are referred to as global registers. These registers may be within the unit (local), in other units (global), in the constant register (in the case of constant values), or within the instruction register (in the case of immediate values). When an instruction is executed, register values are driven onto system busses and routed to the appropriate unit inputs. The unit processes these input values and the result is written to registers within the unit.

The Jazz processor contains 16- and 32-bit data registers. Data and instruction address registers are 16 bits.

Table 4-3. lists the registers contained within each unit, including the width of the register and whether it is local or global.

Table 4-3. it e isters

Unit	Register Name	Size	Description	Type
ACn	mpn.ac0 mpn.ac1 mpn.ac2 mpn.ac3	32	Accumulate Registers	Global
ALn	aln.r0 aln.r1 aln.r2 aln.r3	32	ALU Result Registers	Global
CNT	cnt.c0 cnt.c1 cnt.c2 cnt.c3	16	Counter Registers	Local
CTL	ctl.br	16	Branch Address Register	Local
CTL	ctl.pc	16	Program Counter	Local
CTL	ctl.cnd	1	Condition Register	Local
MPn	mpn.r0 mpn.r1 mpn.r2 mpn.r3	32	Multiply Result Registers	Global
PDM	pdm.x.r0 pdm.y.r0	32	Memory Read Registers (read port x, read port y)	Global
PDM	pdm.x.ar0 pdm.x.ar1 pdm.x.ar2 pdm.x.ar3	16	Address Registers (port x)	Local
PDM	pdm.y.ar0 pdm.y.ar1 pdm.y.ar2 pdm.y.ar3	16	Address Registers (port y)	Local
PDM	pdm.x.ar4 pdm.x.ar5 pdm.x.ar6 pdm.x.ar7	16	Address Registers (port x)	Local
SHn	shn.r3 shn.r2 shn.r1 shn.r0	32	Shift Result Registers	Global
SML	sml.x.r0	32	Memory Read Registers (port x)	Global
SML	sml.x.ar0 sml.x.ar1 sml.x.ar2 sml.x.ar3	16	Address Registers (port x) left memory	Local
SML	sml.x.ar4 sml.x.ar5 sml.x.ar6 sml.x.ar7	16	Address Registers (port x) left memory	Local
SMR	smr.p.r0	32	Memory Read Registers (read port p)	Global
SMR	smr.p.ar0 smr.p.ar1 smr.p.ar2 smr.p.ar3	16	Address Registers (port p) right memory	Local
SMR	smr.p.ar4 smr.p.ar5 smr.p.ar6 smr.p.ar7	16	Address Registers (port p) right memory	Local

Immediate Values

Immediate values are constants specified directly in the instruction. Immediate values are allowed in a number of instruction fields in different instruction formats. The size of the immediate field varies according to the size requirements of the operand it provides.

A single 16-bit immediate field available in each instruction. This field can be used as a constant value within the instruction. At times this value can be referenced as `cst0`.

Saturation

In many applications the simple truncation of the most significant bits is undesirable, making saturation a good alternative to truncation. Where a data value exceeds the maximum (or minimum) value represented by a specific bit width, the value is set to the maximum (or minimum) represented value using saturation. For example, the maximum and minimum values in a 16-bit two's complement representation are +32767 and -32768, respectively. If the result of the addition of two 16-bit numbers is greater than 32767, the result is 32767 with saturation. If the result is less than -32768, the result is -32768 with saturation.

Saturation may also occur when a number represented by a particular bit width is converted to a smaller bit width representation. The shift unit includes both rounding and saturation to support conversion to smaller width representations.

Signed and unsigned saturation are supported in the arithmetic units and the shift unit. Saturation does not affect the CO, OV, and NG condition flags. The ZR flag is calculated after saturation. For packed 8-bit additions, the sign of the source is assumed to be the sign of the saturated result (e.g., signed source data for signed saturate and unsigned for unsigned saturate).

Saturation in the shift unit does not take into account the bits that are shifted out to the left (past bit 63) in a Logical Shift Left (LSL) operation. In the definitions provided in *Table 4-4*, the *sign bit* refers to the sign bit (bit 31) of the source data for LSL and Arithmetic Shift Right (ASR) operations. For Logical Shift Right (LSR) operations the *sign bit* is 0; the source data is always considered positive. In the shift unit, saturation is implemented according to the rules outlined in *Table 4-4*.

Table 4-4. Shift Unit Saturation

Symbol	Definition
S10	The most significant 55 bits of the 64-bit shift result are compared to the sign bit. If all these bits are equal to the sign bit, no saturation occurs. Otherwise, bit 9 is set to the sign bit and the least significant 9 bits of the shift result are set to the complement of the sign bit (e.g., set to the maximum/minimum representable value).
S11	The most significant 54 bits of the 64-bit shift result are compared to the sign bit. If all these bits are equal to the sign bit, no saturation occurs. Otherwise, bit 10 is set to the sign bit and the least significant 10 bits of the shift result are set to the complement of the sign bit (e.g., set to the maximum/minimum representable value).
S12	The most significant 53 bits of the 64-bit shift result are compared to the sign bit. If all these bits are equal to the sign bit, no saturation occurs. Otherwise, bit 11 is set to the sign bit and the least significant 11 bits of the shift result are set to the complement of the sign bit (e.g., set to the maximum / minimum represented value).
S16	The most significant 49 bits of the 64-bit shift result are compared to the sign bit. If all these bits are equal to the sign bit, no saturation occurs. Otherwise, bit 15 is set to the sign bit and the least significant 15 bits of the shift result are set to the complement of the sign bit (e.g., set to the maximum/minimum represented value).
S32	The most significant 33 bits of the 64-bit shift result are compared to the sign bit. If all these bits are equal to the sign bit, no saturation occurs. Otherwise, bit 31 is set to the sign bit and the least significant 31 bits of the shift result are set to the complement of the sign bit (e.g., set to the maximum/minimum represented value).
S8	The most significant 57 bits of the 64-bit shift result are compared to the sign bit. If all these bits are equal to the sign bit, no saturation occurs. Otherwise, bit 7 is set to the sign bit and the least significant 7 bits of the shift result are set to the complement of the sign bit (e.g., set to the maximum/minimum representable value).
S9	The most significant 56 bits of the 64-bit shift result are compared to the sign bit. If all these bits are equal to the sign bit, no saturation occurs. Otherwise, bit 8 is set to the sign bit and the least significant 8 bits of the shift result are set to the complement of the sign bit (e.g., set to the maximum/minimum representable value).
U10	If the sign bit is 0 and any of the most significant 54 bits of the 64-bit shift result are 1 then maximum saturation occurs. Therefore the least significant 10 bits of the shift result are set to 1 (e.g., set to the maximum representable value). If the sign bit is 1, then minimum saturation occurs. Therefore the least significant 10 bits of the shift result are set to 0 (e.g., set to the minimum representable value).
U11	If the sign bit is 0 and any of the most significant 53 bits of the 64-bit shift result are 1 then maximum saturation occurs. Therefore the least significant 11 bits of the shift result are set to 1 (e.g., set to the maximum representable value). If the sign bit is 1, then minimum saturation occurs. Therefore the least significant 11 bits of the shift result are set to 0 (e.g., set to the minimum representable value).
U12	If the sign bit is 0 and any of the most significant 52 bits of the 64-bit shift result are 1 then maximum saturation occurs. Therefore the least significant 12 bits of the shift result are set to 1 (e.g., set to the maximum representable value). If the sign bit is 1, then minimum saturation occurs. Therefore the least significant 12 bits of the shift result are set to 0 (e.g., set to the minimum representable value).
U16	If the sign bit is 0 and any of the most significant 48 bits of the 64-bit shift result are 1 then maximum saturation occurs. Therefore the least significant 16 bits of the shift result are set to 1 (e.g., set to the maximum representable value). If the sign bit is 1, then minimum saturation occurs. Therefore the least significant 16 bits of the shift result are set to 0 (e.g., set to the minimum representable value).
U32	If the sign bit is 0 and any of the most significant 32 bits of the 64-bit shift result are 1 then maximum saturation occurs. Therefore the least significant 32 bits of the shift result are set to 1 (e.g., set to the maximum representable value). If the sign bit is 1, then minimum saturation occurs. Therefore the least significant 32 bits of the shift result are set to 0 (e.g., set to the minimum representable value).
U8	If the sign bit is 0 and any of the most significant 56 bits of the 64-bit shift result are 1 then maximum saturation occurs. Therefore the least significant 8 bits of the shift result are set to 1 (e.g., set to the maximum representable value). If the sign bit is 1, then minimum saturation occurs. Therefore the least significant 8 bits of the shift result are set to 0 (e.g., set to the minimum representable value).

Table 4-4. Shift Unit Saturation (cont'd)

Symbol	Definition
U9	If the sign bit is 0 and any of the most significant 55 bits of the 64-bit shift result are 1 then maximum saturation occurs. Therefore the least significant 9 bits of the shift result are set to 1 (e.g., set to the maximum representable value). If the sign bit is 1, then minimum saturation occurs. Therefore the least significant 9 bits of the shift result are set to 0 (e.g., set to the minimum representable value).

Addition & Subtraction

Normal addition operations are performed the same way for signed and unsigned data. However, use of the condition flags and carry out results must take into account whether the operands are signed or unsigned. Saturation results will also vary based on whether the data is considered signed or unsigned for the operation.

Independent 32-bit arithmetic units cannot be combined together to perform wider width calculations (with the exception of bit-wise operations). There is a 32-bit accumulator unit available for 32-bit accumulate operations. A 64-bit addition operation can also be performed with an arithmetic unit by using the same unit for two consecutive cycles. In the first cycle, the lower 16 bits of the operands are added together and the carry out is stored in the carry register. In the second cycle, the upper 16 bits of the operands are added together with the carry register used as the carry input. This procedure results in correct 32-bit additions and can be extended to 64 bits, 96 bits, and so on. However, the operands cannot be complemented based on the corresponding sign. Saturation cannot be used when doing multi-cycle addition operations.

Table 4-5. Conditions

Code	Condition	Source Unit
CND	Condition Register	ctl
CO	Carry Out	al0
NG	Negative Result	al0
OV	Overflow	al0
TC	Terminal Count	cnt
ZR	Zero Result	al0

Only the arithmetic unit 0 (al0) detects arithmetic conditions. These conditions are listed in *Table 4-6.*

Table 4-6. Arithmetic Conditions

Condition	Mnemonic	Description
Carry Out	CO	Identifies the carry out from the uppermost bit in an addition operation (in other operations, the CO flag is always cleared).
Negative	NG	The result is negative. Overflow is taken into consideration; therefore, the true sign of the result is set in the negative flag. The negative flag is not simply the sign bit of the result, but is the sign bit exclusive with the overflow flag. The NG flag is set as if a signed operation is occurring, even if the actual calculation being performed is unsigned.
Overflow	OV	Identifies that the result of a twos complement addition operation cannot fit in the result register in twos complement notation. The OV flag is set as if a signed operation is occurring, even if the actual calculation being performed is unsigned (in other operations, the OV flag is always cleared).
Zero Result	ZR	The result is zero.

Four arithmetic conditions are detected. A corresponding condition flag is communicated to the control unit for each condition.

The Carry Out and Overflow conditions are only detected for addition operations. These conditions are FALSE for other operations.

The condition flags are set according to the arithmetic operation executing in the current cycle. If there is no arithmetic operation the flags retain their value from the previous operation. In a 16-bit computation (AND, OR, XOR, ADD, CMP, or CND), the condition flags apply to the full 16-bit operation. In an 8-bit computation (ADD8 or CMP8), the condition flags apply to the upper 8-bit operation.

Although only the al0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Signed saturation of signed source operands (all of the same width) occurs if addition results in an overflow. For an n-bit wide calculation, a positive overflow (overflow with a 0 carry out) produces a saturated result of $2^{n-1}-1$ (0 followed by all 1s), and a negative overflow (overflow with a 1 carry out) produces a result of -2^{n-1} (a 1 followed by all 0s).

Unsigned positive saturation of unsigned source operands (all of the same width) occurs if addition results in a carry out of 1 when neither of the inputs is complemented. If this occurs for an n-bit wide calculation, the saturated result is 2^n-1 (all 1s). Unsigned negative saturation of unsigned source operands (all of the same width) occurs if addition results in a carry out of 0 when either of the inputs is complemented. If this occurs for an n-bit wide calculation, the saturated result is 0 (all 0s).

There are six conditions detected within the Jazz processor. The conditions generated by the arithmetic unit (ZR, NG, CO, and OV) are generated in the current cycle. If there is no al0 operation in the current cycle, the values are retained from the last al0 operation. The TC flag is generated by the counter unit in the current cycle. If there is no counter operation in the current cycle, the TC flag is set to the value from the last counter operation. The CND flag is the value of the ctl.cnd register from the last cycle in which it was set.



Chapter 5 Instruction Set Summary

This chapter describes the Jazz processor instruction set used to develop applications. The information provided is organized within the context of the architectural design hierarchy. The processor is composed of core units, Computational Units (CU), and Memory Interface Units (MIU). Within each unit, a series of Micro Operations (MOPs) execute selected tasks.

A VLIW instruction specifies the operations of a μ -task engine for a single clock cycle (instruction cycle). A VLIW instruction consists of a set of Micro Operations (MOPs). Each MOP specifies an individual operation of the μ -task engine. MOPs are executed in parallel (simultaneously). Some operations require the entire task instruction for encoding. The encoding for some MOPs is overlaid with other MOP encoding, making use within a single instruction mutually exclusive.

The individual MOPs are listed alphabetically by mnemonic. The limitations in which MOPs can be used together are also specified. The Jazz processor is configurable; therefore, encoding is not provided for every field within each MOP in this chapter. Refer to *Chapter 4, Assembly Language* for the values for each MOP field.

In This Chapter:

Individual Micro Operations • lxxiv



Individual Micro Operations

This section provides information about each Micro Operation (MOP) in the Jazz processor. The information outlined below is provided for each MOP.

Description

This section provides a brief description of the micro operation.

Reference

This section references the unit to which the micro operation belongs.

Syntax

This section provides the name of each field within the micro operation.

Operand

This section provides the values that can be applied to each field within the argument.



*The names for the **destination** and **source** operands correspond to the ones you assigned to the processor in the Jazz Composer GUI. The Unit Name is designated as **uname**, the memory name as **mname**, and the memory read port as **mrport** for these fields.*

Example

This section provides a sample argument for the micro operation.

ACA

Accumulator Accumulate

Description

An ACA operation causes the accumulator result registers specified in the destination field to be loaded with the sum of their own value, and the value in the multiplier result registers with which they are paired (e.g., the same register number).

Reference

Computational Unit (MAC_32).

Syntax

ACA Destination ;

Operand

Destination:

uname.ac1:uname.ac0 | uname.ac3:uname.ac2

The Destination field must be two of the result registers within the accumulator unit.

Example

ACA mp0.ac1:mp0.ac0 ;

ACZ

Accumulator Zero

Description

An ACZ operation loads the accumulator result registers specified in the destination field with zero.

Reference

Computational Unit (MAC_32).

Syntax

ACZ Destination ;

Operand

Destination:

uname.ac1:uname.ac0 | uname.ac3:uname.ac2

The Destination field must be two of the result registers within the accumulator unit.

Example

```
ACZ mp0.ac1:mp0.ac0 ;
```

ADD

Addition

Description

An ADD operation specifies a 32-bit addition operation generating a 32-bit result. The ADD operation is a 32-bit addition of the left and right operands, and the input carry bit. The result is a 32-bit value and a carry out bit. The 32-bit result is stored in the specified result register. The carry out and status bits of arithmetic unit 0 (al0) are used to set the carry out condition flag.

Reference

Computational Unit (ALU_32).

Syntax

ADD Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register | 4-bit immediate

The R.operand field specifies the right input data value, and may be a 32-bit global register or a 4-bit immediate (zero extended to 32 bits).

L.comp:

F

The L.comp field specifies that the left input is not complimented.

R.comp:

T | F | N | P

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the ADD operation, the right input can be complimented (T), not complimented (F), complimented if it is negative (N), or complimented if it is positive (P).

CarryIn:

CL | CO

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented, and 0 otherwise. CO specifies that the input carry value is the carry register, which is set to the carry out generated by the same arithmetic unit in the previous cycle. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

S8 | S16 | S32 | U8 | U16 | U32 | NS

The Saturate field specifies the kind of saturation that is performed. For saturation with an ADD operation, the source operands are always assumed to be signed, even when saturating to an unsigned result. Saturation is supported to a result width smaller than the source operands.

Cmask:

ctl.cst1

The Cmask field is not used in the ADD operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

ADD al0.r0 zero pdm.rx.r0 F F CL NS ctl.cst1 ;

ADD16

Addition (16-Bit)

Description

The ADD16 operation specifies a packed pair of 16-bit additions generating two packed 16-bit results. The ADD16 specifies a packed 16-bit operation that treats the two 32-bit inputs as a packed pair of 16-bit values. The upper and lower 16 bits form the first and second values respectively. The 32-bit result contains the packed pair of 16-bit result values. The carry out and status bits on arithmetic unit 0 (al0) are from the upper 16-bit operation. The input compliment operations and carry in selection support the packed ADD16 operation. If the compliment operation or carry in selection is specified as dependent on the sign bit, the sign bits of the upper and lower 16-bit (or the four 8-bit) values are independently processed.

Reference

Computational Unit (ALU_32).

Syntax

ADD16 Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0 | uname.r1 | uname.r2 | uname.r3

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented.

R.comp:

T | F | N | P

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the ADD16 operation, the right input can be complimented (T), not complimented (F), complimented if it is negative (N), or complimented if it is positive (P).

CarryIn:

CL | CO

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented, and 0 otherwise. CO specifies that the input carry value is the carry register, which is set to the carry out generated by the same arithmetic unit in the previous cycle. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. While AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the ADD16 operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the ADD16 operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

ADD16 al0.r0 pdm.rx.r0 al0.r2 F T CL NS ctl.cst1 ;

ADD5

Addition (5 Bits)

Description

The ADD5 operation specifies a 5-bit addition generating a single 5-bit result. The operation ignores the upper 11 bits of the input operands. The carry out and status bits of the ADD5 operation on arithmetic unit 0 (al0) are from the 5-bit operation. The 16-bit result of an ADD5 operation contains the 5-bit result with the upper 11 bits set to zero. Even if the result is negative (e.g., the NG flag is set), the upper bits are set to zero and never sign extended.

Reference

Computational Unit (ALU_32).

Syntax

ADD5 Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register | 5-bit immediate

The R.operand field specifies the right input data value, and may be a 32-bit global register or a 5-bit immediate (zero extended to 32 bits). The upper 27 bits are ignored for the ADD5 operation.

L.comp:

F

The L.comp field specifies that the left input is not complimented.

R.comp:

T | F | N | P

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the ADD5 operation, the right input can be complimented (T), not complimented (F), complimented if it is negative (N), or complimented if it is positive (P).

CarryIn:

CL | CO

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented, and 0 otherwise. CO specifies that the input carry value is the carry register, which is set to the carry out generated by the same arithmetic unit in the previous cycle. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the ADD5 operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the ADD5 operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

ADD5 al0.r0 al0.r2 pdm.rx.r0 F F CL NS ctl.cst1 ;

ADD8

Addition (8-Bit)

Description

The ADD8 operation specifies a packed quad of 8-bit additions generating four packed 8-bit results. The ADD8 operator specifies a packed 8-bit operation that treats each of the two 32-bit inputs as a packed quad of 8-bit values. The 32-bit result contains the packed quad of 8-bit result values. The carry out and status bits of the ADD8 operation on arithmetic unit 0 (al0) are from the uppermost 8-bit operation. The input compliment operations and carry in selection support the packed ADD8 operation.

Reference

Computational Unit (ALU_32).

Syntax

ADD8 Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0 | uname.r1 | uname.r2 | uname.r3

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented.

R.comp:

T | F | N | P

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the ADD8 operation, the right input can be complimented (T), not complimented (F), complimented if it is negative (N), or complimented if it is positive (P).

CarryIn:

CL | CO

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented, and 0 otherwise. CO specifies that the input carry value is the carry register, which is set to the carry out generated by the same arithmetic unit in the previous cycle. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

S8 | U8 | NS

The Saturate field specifies the kind of saturation that is performed. For saturation with an ADD8 operation, the source operands are always assumed to be signed when signed saturate is specified, and unsigned when unsigned saturate is specified. Saturation is supported to a result width smaller than the source operands.

Cmask:

ctl.cst1

The Cmask field is not used in the ADD8 operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

ADD8 al0.r0 pdm.rx.r0 al0.r2 F F CL NS ctl.cst1 ;

ADDF

ADD Flags

Description

An ADDF operation copies the contents of the flag registers into one of the global result registers in the ALU. The flag register status is required at the start of interrupt service routines to save the processor state.

Reference

Computational Unit (ALU_32).

Syntax

ADDF Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.status

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

zero

The L.operand field specifies the left input data value, and is always zero.

R.operand:

uname.status

The R.operand field specifies the status of the ALU flag. The status can be ZR (zero), NG (negative), CO (carry out), or OV (overflow).

L.comp:

F

The L.comp field specifies that the left input is not complimented.

R.comp:

F

The R.comp field specifies that the right input is not complimented.

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented, and 0 otherwise. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies that no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the ADD operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

```
ADDF al0.r0 zero al0.status F F CL NS ctl.cst1 ;
```

ADLUI

Immediate Load ADLU

Description

The ADLUI operation specifies an immediate load of an output result.

Reference

Core Unit (ADLU).

Syntax

ADLUI Destination L.operand ;

Operand

Destination:

uname.r0 | *uname.r1* | *uname.r2* | *uname.r3*

The Destination field specifies the destination register of the result. The value must be one of the result registers within the ADLU.

L.operand:

32-bit register

The L.operand field specifies the 32-bit global register for the left input data value.

Example

```
ADLUI adlu.r0 al0.r0 ;
```

AND

And (Bitwise)

Description

The AND operation is a bit wise 32-bit operation generating a 32-bit result.

Reference

Computational Unit (ALU_32).

Syntax

AND Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0 | *uname.r1* | *uname.r2* | *uname.r3*

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | **zero**

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register | **4-bit immediate**

The R.operand field specifies the right input data value, and may be a 32-bit global register or a 4-bit immediate (zero extended to 32 bits).

L.comp:

T | **F**

The L.comp field specifies whether a bit-wise compliment is applied to the input values. For the AND operation, the left input can be complimented (T) or not complimented (F).

R.comp:

T | **F**

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the AND operation, the right input can be complimented (T) or not complimented (F).

CarryIn:

CL | **CO**

The CarryIn field specifies the input carry value for addition operations. The value in this field is ignored for logical operations.

Saturate:

NS

The Saturate field specifies the kind of saturation that is performed. For the AND operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the AND operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

```
AND al0.r0 al0.r3 0x0 F F CL NS ctl.cst1 ;
```

ASR

Arithmetic Shift Right

Description

An ASR operation specifies an arithmetic shift right operation filled with the sign bit (bit 63) of the source data. The ASR operation cannot be specified when zero extension of the input operand is specified.

Reference

Computational Unit (SHIFT_32).

Syntax

ASR Destination Source Amount Rounding Saturate ;

Operand

Destination:

uname.r1:uname.r0 | uname.r3:uname.r2

The Destination field must be two of the result registers within the shift unit.

Source:

32-bit register:32-bit register | 0:32-bit register | 0

The Source field specifies the input data to be shifted. For an ASR operation, the input data is either a 64-bit signed value, a 32-bit signed value or 0. Whenever the concatenation specifies two 32-bit global registers, the input format is 64-bit signed. Whenever the concatenation specifies one 32-bit global register, the input format is 32-bit signed.

Amount:

constant value | 32-bit register | zero

The Amount field specifies the number of bits the data is shifted. The least significant 6 bits (unsigned) of a register or constant are used. The shift amount must be in the range of 0 to 63.

Rounding:

R | NR

The Rounding field specifies whether the result is rounded (R) or not rounded (NR).

Saturate:

S32 | S16 | S12 | S10 | S11 | S9 | S8 | U32 | U16 | U12 | U11 | U10 | U9 | U8 | NS

The Saturate field specifies the type of saturation (if any) that is performed. For an ASR operation, the source data is always considered to be a 64-bit signed value. Refer to *Chapter 4, Assembly Language* for detailed information about saturation and the valid entries for this field.

Example

ASR sh0.r1:sh0.r0 pdm.rx.r0:al0.r2 al0.r3 NR NS ;



*In an ASR operation of **n** bits, the **n** least significant bits of the source are shifted out and lost. Rounding reduces the mathematical error introduced by the loss of these bits.*

BR

Positive Conditional Branch

Description

A BR operation jumps to the branch address only if the condition register is true. The task instruction immediately following a BR is always executed unless the instruction is marked for conditional execution, and the condition register is set accordingly.

The condition register (ctl.cnd) is contained within the controller unit. The condition register is tested at the start of the task execution instruction (e.g., the condition reflects the results of previous μ -task instructions) for the BR operation.

The BR operation updates the program counter with the content of the branch address register (ctl.br) only if the condition register is set appropriately.

Reference

Core Unit (Branch Control).

Syntax

BR ;

Example

BR ;



The following branch control operations cannot be used in sequence: BR, CALL, ET, INTR, NBR, RET, RETE, UBR or UET.

The following operations cannot be used simultaneously with any of the branch control instructions: DINT, EINT, PUSH or POP.

CALL

Call Sub-Routine

Description

A CALL operation unconditionally jumps to the branch address. The instruction immediately following a CALL is always executed unless the instruction is marked for conditional execution, and the condition register is set accordingly.

The CALL operation causes the next program counter to be saved to the hardware stack. The program counter is also updated with the contents of the branch address register (ctl.br). If the instruction also contains the Set Branch Register (SBR) micro operation, the program counter is updated directly with the new branch register value.

Reference

Core Unit (Branch Control).

Syntax

CALL ;

Example

CALL ;



The following branch control operations cannot be used in sequence: BR, CALL, ET, INTR, NBR, RET, RETE, UBR or UET.

The following operations cannot be used simultaneously with any of the branch control instructions: DINT, EINT, PUSH or POP.

The CALL operation is only available to processors configured with a hardware stack.

CE

Positive Conditional Execute

Description

A CE operation executes the task instruction only if the condition register is true. The condition register (ctl.cnd) is contained within the controller unit. The condition register is tested at the start of the task execution instruction (e.g., the condition reflects the results of previous μ -task instructions) for the CE operation.

The CE operation causes the remaining operations in the task instruction to be executed only if the condition register is set appropriately. If the task instruction is predicated upon this operation, none of the other operations in the task instruction are completed (e.g., no result registers are modified).

Reference

Core Unit (Branch Control).

Syntax

CE ;

Example

CE ;



The following operations cannot be used simultaneously with any of the branch control instructions: DINT, EINT, PUSH, or POP.

CMP

Compare (32 Bits)

Description

A CMP operation specifies a 32-bit comparison operation. If the 32-bit input values are equal, the 32-bit result is *1*; otherwise it is *0*.

Reference

Computational Unit (ALU_32).

Syntax

CMP Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0 | *uname.r1* | *uname.r2* | *uname.r3*

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | **zero**

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

T | **F**

The L.comp field specifies whether a bit-wise compliment is applied to the input values. For the CMP operation, the left input can be complimented (T) or not complimented (F).

R.comp:

T | **F**

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the CMP operation, the right input can be complimented (T) or not complimented (F).

CarryIn:

CL | **CO**

The CarryIn field specifies the input carry value for addition operations. CL specifies *1* for carry in if the right operand is complemented, and *0* otherwise. CO specifies that the input carry value is the carry register, which is set to the carry out generated by the same arithmetic unit in the previous cycle. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the CMP operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the CMP operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

```
CMP al0.r0 al0.r0 pdm.rx.r0 F F CL NS ctl.cst1 ;
```


CMP16

Compare (16 Bits)

Description

A CMP16 operation specifies a packed pair of 16-bit comparison operations, generating two packed 16-bit results.

Reference

Computational Unit (ALU_32).

Syntax

CMP16 Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0 | *uname.r1* | *uname.r2* | *uname.r3*

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | **zero**

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

T | **F**

The L.comp field specifies whether a bit-wise compliment is applied to the input values. For the CMP16 operation, the left input can be complimented (T) or not complimented (F).

R.comp:

T | **F**

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the CMP16 operation, the right input can be complimented (T) or not complimented (F).

CarryIn:

CL | **CO**

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented, and 0 otherwise. CO specifies that the input carry value is the carry register, which is set to the carry out generated by the same arithmetic unit in the previous cycle. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the CMP16 operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the CMP16 operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

```
CMP16 al0.r0 al0.r0 pdm.rx.r0 F F CL NS ctl.cst1 ;
```

CMP8

Compare (8 Bits)

Description

A CMP8 operation specifies a packed quad of 8-bit comparison operations, generating four packed 8-bit results.

Reference

Computational Unit (ALU_32).

Syntax

CMP8 Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

T | F

The L.comp field specifies whether a bit-wise compliment is applied to the input values. For the CMP8 operation, the left input can be complimented (T) or not complimented (F).

R.comp:

T | F

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the CMP8 operation, the right input can be complimented (T) or not complimented (F).

CarryIn:

CL | CO

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented, and 0 otherwise. CO specifies that the input carry value is the carry register, which is set to the carry out generated by the same arithmetic unit in the previous cycle. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the CMP8 operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the CMP8 operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

```
CMP8 al0.r0 al0.r0 pdm.rx.r0 F F CL NS ctl.cst1 ;
```

CND

Condition (al0 Only)

Description

A CND operation specifies a 32-bit addition operation generating the condition codes. These codes, together with the TC flag from the counter unit, are compared with the condition mask (specified by Cmask) to generate a condition value. If any of the positive or negative conditions specified in the condition mask match, the condition value is 1; otherwise it is 0. This condition value is written as the 32-bit result. Only AL0 can perform this operation.

Reference

Computational Unit (ALU_32).

Syntax

CND Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented.

R.comp:

T | F | N | P

The R.comp field specifies whether the right input is complimented (T), not complimented (F), complimented if it is negative (N), or complimented if it is positive (P).

CarryIn:

CL | CO

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented, and 0 otherwise. CO specifies that the input carry value is the carry register, which is set to the carry out generated by the same arithmetic unit in the previous cycle. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the CND operation, no saturation is performed.

Cmask:

constant value | zero

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

CND al0.r2 mp0.r0 al0.r2 F F CL NS 0x200 ;

*CRI

Constant Register Immediate

Description

A CRI operation writes immediate values to the constant register. The CRI comprises an entire task instruction.

Constant registers are loaded beginning with the constant register selected by the index. The number of constant registers that are loaded depends upon the number of 16-bit immediate values contained within the instruction width of the processor. To maintain a value in a constant register, it must be reloaded with the same value.

Reference

Core Unit (Constant Register).

Syntax

CRI Index *datan* ;

Operand

Index:

The value in the Index field corresponds to the start location of the initial Immediates when they are loaded into the constant register.

Datan:

16-bit immediate value

The *Datan* field specifies the values to be stored in the constant registers **cstn**, where *n* is a value from one (1) through fourteen (14).

Example

CRI 0x0 0x1 0x2 0x3 0x4 0x5 0x6 0x7 0x8 0x9 0xa 0xb 0xc 0xd 0xe ;



The CRI is a special instruction that makes use of the entire VLIW and cannot be used simultaneously with any other special instruction or micro operation.

CTD

Counter Decrement

Description

A CTD operation sets the destination register to the source value minus 1. An instruction containing a CTD operation must also include an SCR operation that sets the condition register to TC or !TC.

A counter unit operation specifies a function performed by the counter unit. A task instruction may contain only one counter unit operation.

Reference

Core Unit (Counter Register).

Syntax

CTD Destination Source ;

Operand

Destination:

uname.r0 | uname.r1 | uname.r2 | uname.r3

The Destination field must contain one of the four 16-bit counter registers within the counter unit.

Source:

constant value | 32-bit register | uname.r0 | uname.r1 | uname.r2 | uname.r3

The Source field specifies the input (set) data value. If this field contains a counter register, it must match the register in the destination field.

Example

```
CTD cnt.c0 cnt.c0 ;                                <-- simple decrement
```



The following operations cannot be used simultaneously with any of the counter instructions: PR_ROT, PR_WR, SBR, SCR, SCTL, VPM or WPR.

When the CTD micro operation uses a 32-bit register for the Source operand, the shared global bus must be used which may conflict with other micro operations that use this resource. When the CTD micro operation uses a constant value for the Source operand, the shared constant bus must be used, which may conflict with other operations that use this resource.

CTS

Counter Set

Description

A CTS operation sets the destination register to the source value. An instruction containing a CTS operation must also include an SCR operation that sets the condition register to !TC.

Reference

Core Unit (Counter Register).

Syntax

CTS Destination Source ;

Operand

Destination:

uname.r0 | uname.r1 | uname.r2 | uname.r3

The Destination field must contain one of the four 16-bit counter registers within the counter unit.

Source:

constant value | 32-bit register | uname.r0 | uname.r1 | uname.r2 | uname.r3

The Source field specifies the input (set) data value. If this field contains a counter register, it must match the register in the destination field.

Example

```
CTS cnt.c1 6 ;                                <-- set to 6
```



The following operations cannot be used simultaneously with any of the counter instructions: PR_ROT, PR_WR, SBR, SCR, SCTL, VPM or WPR. When the CTS micro operation uses a 32-bit register for the Source operand, the shared global bus must be used which may conflict with other micro operations that use this resource. When the CTS micro operation uses a constant value for the Source operand, the shared constant bus must be used which may conflict with other micro operations that use this resource.

EQ

Relational Equal

Description

An EQ operation specifies a 32-bit comparison operation. If the 32-bit values are equal, the result is 1; otherwise, it is 0.

Reference

Computational Unit (ALU_32).

Syntax

EQ Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the EQ operation, no saturation is performed.

Cmask:

0x1000

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

EQ al0.r0 pdm.rx.r0 al0.r2 F T CL NS 0x1000 ;

ET

Positive Conditional End Task

Description

An ET operation ends the current μ -task if the condition register is true. The task instruction immediately following an ET is always executed unless the instruction is marked for conditional execution, and the condition register is set accordingly.

The condition register (ctl.cnd) is contained within the controller unit. The condition register is tested at the start of the task execution instruction (e.g., the condition reflects the results of previous μ -task instructions) for the ET operation.

The ET operation causes the next μ -task from the task queue to be initiated (program counter and address registers are updated) if the condition register is true.

Reference

Core Unit (Branch Control).

Syntax

ET ;

Example

ET ;



The following branch control operations cannot be used in sequence: BR, CALL, ET, INTR, NBR, RET, RETE, UBR or UET.

The following operations cannot be used simultaneously with any of the branch control instructions: DINT, EINT, PUSH or POP.

GT

Relational Greater Than

Description

A GT operation specifies a signed 32-bit comparison operation. If the signed 32-bit L.operand is greater than the signed 32-bit R.operand, the result is 1; otherwise it is 0.

Reference

Computational Unit (ALU_32).

Syntax

GT Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0 | *uname.r1* | *uname.r2* | *uname.r3*

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | **zero**

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

T

The L.comp field specifies that the left input is complimented (T).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the GT operation, no saturation is performed.

Cmask:

0x9800

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

GT al0.r2 mp0.r0 sh0.r1 F T CL NS 0x9800 ;

GTE

Relational Greater Than or Equal

Description

A GTE operation specifies a signed 32-bit comparison operation. If the signed 32-bit L.operand is greater or equal to the signed 32-bit R.operand, the result is 1; otherwise it is 0.

Reference

Computational Unit (ALU_32).

Syntax

GTE Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the GTE operation, no saturation is performed.

Cmask:

ox8

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

GTE al0.r2 mp0.r0 sh0.r2 F T CL NS ox8 ;

HTQ

High Priority Task Queue

Description

The HTQ operation inserts a μ -task into the high priority task queue of a task engine. The high priority task queue can store the number of tasks configured by the user. When a task queue is full, queue task instructions specifying the engine containing the full queue will have no effect (e.g., the task to be queued is discarded and lost).

The Q-Bus is a shared resource among all engines, so it is possible that a higher priority μ -task engine has control of the Q-Bus when another engine also has a μ -task queue operation to execute. In this case, the lower priority engine may need to stall, thereby ceasing all program execution until the Q-Bus is available. There are buffers in place to limit the occurrence of stalled engines.

Reference

Core Unit (Task Queue).

Syntax

HTQ Task.addr Parameter ;

Operand

Task.addr:

constant value | 32-bit register

The Task.addr field specifies the address of the μ -task being queued. The upper four (4) bits of the address specify the task engine, IO engine, or IO module. The lower 12 bits of the task.addr field specify the local task address in instruction memory.

Parameter:

constant value | 32-bit register

The Parameter field specifies a 16-bit parameter that is queued with the μ -task. The parameter value is loaded into the lowest number read address registers of the engine (pdm.rax0, sml.rax0, and smr.rap0) when the task is initiated.

Example

HTQ 0x37A2 a10.r0 ;



The HTQ operation cannot be used simultaneously with any private data memory WX port write operation.

When the HTQ operation uses a 32-bit register for the Task.addr operand, the shared global bus must be used which may conflict with other operations that use this resource.

INTR

Call Sub-Routine; Disable Interrupts

Description

A INTR operation unconditionally jumps to the immediate address. The instruction immediately following an INTR is always executed unless the instruction is marked for conditional execution, and the condition register is set accordingly.

The INTR operation causes the next program counter to be saved to the hardware stack. The program counter is also updated with the immediate address, and the interrupt mask register is set.

Reference

Core Unit (Branch Control).

Syntax

INTR Task.addr ;

Operand

Task.addr:

16-bit immediate address

The Task.addr field specifies...

Example

INTR ;



The following branch control operations cannot be used in sequence: BR, CALL, ET, INTR, NBR, RET, RETE, UBR or UET. The following operations cannot be used simultaneously with any of the branch control instructions: DINT, EINT, PUSH or POP.

The INTR operation is only available to processors configured with a hardware stack.

ITQ

Interrupt Task Queue

Description

The ITQ operation inserts a μ -task into the interrupt task queue of a task engine. The interrupt task queue can store the number of tasks configured by the user. When a task queue is full, queue task instructions specifying the engine containing the full queue will have no effect (e.g., the task to be queued is discarded and lost).

The Q-Bus is a shared resource among all engines, so it is possible that a higher priority μ -task engine has control of the Q-Bus when another engine also has a μ -task queue operation to execute. In this case, the lower priority engine may need to stall, thereby ceasing all program execution until the Q-Bus is available. There are buffers in place to limit the occurrence of stalled engines.

Reference

Core Unit (Task Queue).

Syntax

ITQ Task.addr Parameter ;

Operand

Task.addr:

constant value | 32-bit register

The Task.addr field specifies the address of the μ -task being queued. The upper four (4) bits of the address specify the task engine, IO engine, or IO module. The lower 12 bits of the task.addr field specify the local task address in instruction memory.

Parameter:

constant value | 32-bit register

The Parameter field specifies a 16-bit parameter that is queued with the μ -task. The parameter value is loaded into the lowest number read address registers of the engine (pdm.rax0, sml.rax0, and smr.rap0) when the task is initiated.

Example

ITQ 0x37A2 a10.r0 ;



The ITQ operation cannot be used simultaneously with any private data memory WX port write operation.

When the ITQ operation uses a 32-bit register for the Task.addr operand, the shared global bus must be used which may conflict with other operations that use this resource.

JEXP

Exponent Extraction

Description

A JEXP operation specifies an exponent extraction. The 32-bit result is the lowest power of two that is greater than the 32-bit R.operand.

Reference

Computational Unit (ALU_32).

Syntax

JEXP Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0 | *uname.r1* | *uname.r2* | *uname.r3*

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | **zero**

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented.

R.comp:

T | **F** | **N** | **P**

The R.comp field specifies whether the right input is complimented (T), not complimented (F), complimented if it is negative (N), or complimented if it is positive (P).

CarryIn:

CL | **CO**

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented, and 0 otherwise. CO specifies that the input carry value is the carry register, which is set to the carry out generated by the same arithmetic unit in the previous cycle. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the JEXP operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the JEXP operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

```
JEXP al0.r0 zero al0.r0 F F CL NS ctl.cst1 ;
```

LSL

Logical Shift Left

Description

An LSL operation specifies a logical shift left operation filled with zeros. The LSL operation cannot be specified when zero extension of the input operand is specified.

Reference

Computational Unit (SHIFT_32).

Syntax

LSL Destination Source Amount Rounding Saturate ;

Operand

Destination:

uname.r1:uname.r0 | uname.r3:uname.r2

The Destination field must be two of the result registers within the shift unit.

Source:

32-bit register:32-bit register | 0:32-bit register | 0

The Source field specifies the input data to be shifted. For an LSL operation, the input data is either a 64-bit signed value, a 32-bit signed value or 0. Whenever the concatenation specifies two 32-bit global registers, the input format is 64-bit signed. Whenever the concatenation specifies one 32-bit global register, the input format is 32-bit signed.

Amount:

constant value | 32-bit register | zero

The Amount field specifies the number of bits the data is shifted. The least significant 6 bits (unsigned) of a register or constant are used. The shift amount must be in the range of zero to 63.

Rounding:

R | NR

The Rounding field specifies whether the result is rounded (R) or not rounded (NR).

Saturate:

S32 | S16 | S12 | S10 | S11 | S9 | S8 | U32 | U16 | U12 | U11 | U10 | U9 | U8 | NS

The Saturate field specifies the type of saturation (if any) that is performed. For an LSL operation, the source data is always considered to be a 64-bit signed value. Refer to *Chapter 4, Assembly Language* for detailed information about saturation and the valid entries for this field.

Example

```
LSL sh0.r1:sh0.r0 pdm.rx.r0:al0.r2 al0.r3 NR NS ;
```



Rounding has no effect when specified in conjunction with an LSL operation.

LSR

Logical Shift Right

Description

An LSR operation specifies a logical shift right operation filled with zeros. The LSR operation cannot be specified when sign extension of the input operand is specified.

Reference

Computational Unit (SHIFT_32).

Syntax

LSR Destination Source Amount Rounding Saturate ;

Operand

Destination:

uname.r1:uname.r0 | uname.r3:uname.r2

The Destination field must be two of the result registers within the shift unit.

Source:

32-bit register:32-bit register | 0:32-bit register | 0

The Source field specifies the input data to be shifted. For an LSR operation, the input data is either a 64-bit unsigned value, a 32-bit unsigned value or 0. Whenever the concatenation specifies two 32-bit global registers, the input format is 64-bit unsigned. Whenever the concatenation specifies one 32-bit global register, the input format is 32-bit unsigned.

Amount:

constant value | 32-bit register | zero

The Amount field specifies the number of bits the data is shifted. The least significant 6 bits (unsigned) of a register or constant are used. The shift amount must be in the range of 0 to 63.

Rounding:

R | NR

The Rounding field specifies whether the result is rounded (R) or not rounded (NR).

Saturate:

S32 | S16 | S12 | S10 | S11 | S9 | S8 | U32 | U16 | U12 | U11 | U10 | U9 | U8 | NS

The Saturate field specifies the type of saturation (if any) that is performed. For an LSR operation, the source data is always considered to be a 64-bit unsigned value. Refer to *Chapter 4, Assembly Language* for detailed information about saturation and the valid entries for this field.

Example

```
LSR sh0.r1:sh0.r0 zero:al0.r2 0x1f NR NS ;
```



*In an LSR operation of **n** bits, the **n** least significant bits of the source are shifted out and lost. Rounding reduces the mathematical error introduced by the loss of these bits.*

LSS

Load Sum Sign Extended

Description

An LSS operation specifies that the sum of the address (src.ar) and the increment value (inc) is sign extended to 32 bits and loaded into the read register. The 16 bits of data read is always aligned with the lower half of the read register.

Reference

Memory Interface Unit.

Syntax

LSS [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

mname.mport.rn | zero

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the LSS operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource. If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

LSU

Load Sum Unsigned

Description

An LSU operation specifies that the sum of the address (src.ar) and the increment value (inc) is zero extended to 32 bits and loaded into the read register. The 16 bits of data read is always aligned with the lower half of the read register.

Reference

Memory Interface Unit.

Syntax

LSU [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

mname.mport.rn | zero

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the LSU operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource.

If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

LT

Relational Less Than

Description

An LT operation specifies a signed 32-bit comparison operation. If the signed 32-bit L.operand is less than the signed 32-bit R.operand, the result is 1; otherwise, it is 0.

Reference

Computational Unit (ALU_32).

Syntax

LT Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the LT operation, no saturation is performed.

Cmask:

0x800

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

LT al0.r2 mp0.r0 sh0.r2 F T CL NS 0x800 ;

LTE

Relational Less Than or Equal

Description

An LTE operation specifies a signed 32-bit comparison operation. If the signed 32-bit L.operand is less than or equal to the signed 32-bit R.operand, the result is *1*; otherwise, it is *0*.

Reference

Computational Unit (ALU_32).

Syntax

LTE Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies *1* for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the LTE operation, no saturation is performed.

Cmask:

0x1800

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

LTE al0.r2 mp0.r0 sh0.r2 F T CL NS 0x1800 ;

LWD

Load Write Data

Description

An LWD operation writes the data on the write port of this memory (the data value specified in the src.dat field of a memory write operation to the same memory) to the read register of this memory port.

For example, to write a 32-bit global data value to the pdm.rx register the data value must be specified as the src.dat in a write operation of pdm. If also writing this value to memory is not desired, then an SWA instruction can be used as the write operation that specifies this source data. The SWA can use a null address register if no address calculation is desired. The SWA with a null address register does not count as one of the five memory operations. The address or increment registers are not modified by this instruction.

Reference

Memory Interface Unit.

Syntax

LWD Dest.r Src.dat Src.a Src.inc ;

Operand

Dest.r:

***mname.mrport.r0* | zero**

The Dest.r field specifies the destination read register. It must be one of the read registers in the private or shared memory units, or zero.

Src.dat:

32-bit register

The Src.dat field specifies a 32-bit register as the source for the data written to memory.

Src.a:

zero

The Src.a field specifies the source of the address value. It must be zero or one of the address registers associated with the read port.

Src.inc:

zero

The Src.inc field specifies the source of the increment value. It must be zero.

Example

```
LWD pdm.rx.r0 al0.r1 zero zero ;
```



If local address registers are specified for the dest.a and src.a fields, they must share the same register number.

MASK

Mask

Description

A MASK operation specifies a 32-bit mask operation, generating a 32-bit result. This operation masks off the upper bits of the left operand to zero, retaining 0 to 31 of the lower bits. The number of bits retained is specified by the lower 5 bits of the right operand.

Reference

Computational Unit (ALU_32).

Syntax

MASK Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register | 5-bit immediate

The R.operand field specifies the right input data value, and may be a 32-bit global register or a 5-bit immediate (zero extended to 32 bits). The upper 27 bits are ignored for the MASK operation.

L.comp:

T | F

The L.comp field specifies whether a bit-wise compliment is applied to the input values. For the MASK operation, the left input can be complimented (T) or not complimented (F).

R.comp:

T | F

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the MASK operation, the right input can be complimented (T) or not complimented (F).

CarryIn:

CL | CO

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented, and 0 otherwise. CO specifies that the input carry value is the carry register, which is set to the carry out generated by the same arithmetic unit in the previous cycle. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation that is performed. For the MASK operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the MASK operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

```
MASK al0.r0 al0.r2 0xc F F CL NS ctl.cst1 ;
```

MAX

Maximum

Description

A MAX operation specifies a signed 32-bit comparison operation. If the signed 32-bit L.operand is greater than the signed 32-bit R.operand, the result is the L.operand; otherwise, it is the R.operand.

Reference

Computational Unit (ALU_32).

Syntax

MAX Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the MAX operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the MAX operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

```
MAX al0.r2 mp0.r0 sh0.r2 F T CL NS ctl.cst1 ;
```

MIN

Minimum

Description

A MIN operation specifies a signed 32-bit comparison operation. If the signed 32-bit L.operand is less than the signed 32-bit R.operand, the result is the L.operand; otherwise, it is the R.operand.

Reference

Computational Unit (ALU_32).

Syntax

MIN Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the MIN operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the MIN operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

```
MIN al0.r2 mp0.r0 sh0.r2 F T CL NS ctl.cst1 ;
```


MPS

Multiply Signed

Description

An MPS instruction specifies a signed multiply operation to be performed by the multiply unit.

The multiply unit performs 32-bit by 32-bit integer multiplication operations, generating a 64-bit result. Both the left and right operands are treated as signed values.

Reference

Computational Unit (MAC_32).

Syntax

MPS Destination L.operand R.operand ;

Operand

Destination:

uname.r1:uname.r0 | uname.r3:uname.r2

The Destination of a multiply operation must be two of the result registers within the multiply unit.

L.operand:

32-bit register

The L.operand field specifies the left input data value. This is a 32-bit global register.

R.operand:

32-bit register

The R.operand field specifies the right input data value. This is a 32-bit global register.

Example

MPS mp0.r1:mp0.r0 mp0.r2 pdm.rx.r0 ;

MPU

Multiply Unsigned

Description

An MPU instruction specifies an unsigned multiply operation to be performed by the multiply unit.

The multiply unit performs 32-bit by 32-bit integer multiplication operations, generating a 64-bit result. Both the left and right operands are treated as unsigned values.

Reference

Computational Unit (MAC_32).

Syntax

MPS Destination L.operand R.operand ;

Operand

Destination:

uname.r1:uname.r0 | uname.r3:uname.r2

The Destination of a multiply operation must be one of the two 64-bit result registers within the multiply unit.

L.operand:

32-bit register

The L.operand field specifies the left input data value. This is a 32-bit global register.

R.operand:

32-bit register

The R.operand field specifies the right input data value. This is a 32-bit global register.

Example

MPU mp0.r1:mp0.r0 mp0.r2 pdm.rx.r0 ;

MPW

Multiply Written

Description

An MPW instruction specifies that the source operands be written directly to the result register in the multiply unit.

The multiply unit performs 32-bit by 32-bit integer multiplication operations, generating a 64-bit result.

Reference

Computational Unit (MAC_32).

Syntax

MPW Destination L.operand R.operand ;

Operand

Destination:

uname.r1:uname.r0 | uname.r3:uname.r2

The Destination of a multiply operation must be one of the two 64-bit result registers within the multiply unit.

L.operand:

32-bit register

The L.operand field specifies the left input data value. This is a 32-bit global register.

R.operand:

32-bit register

The R.operand field specifies the right input data value. This is a 32-bit global register.

Example

```
MPW mp0.r1:mp0.r0 mp0.r2 pdm.rx.r0 ;
```

NBR

Negative Conditional Branch

Description

An NBR operation jumps to the branch address only if the condition register is false. The task instruction immediately following an NBR is always executed unless the instruction is marked for conditional execution, and the condition register is set accordingly.

The condition register (ctl.cnd) is contained within the controller unit. The condition register is tested at the start of the task execution instruction (e.g., the condition reflects the results of previous μ -task instructions) for the NBR operation.

The NBR operation updates the program counter with the content of the branch address register (ctl.br) only if the condition register is set appropriately.

Reference

Core Unit (Branch Control).

Syntax

NBR ;

Example

NBR ;



The following branch control operations cannot be used in sequence: BR, CALL, ET, INTR, NBR, RET, RETE, UBR or UET.

The following operations cannot be used simultaneously with any of the branch control instructions: DINT, EINT, PUSH or POP.

NCE

Negative Conditional Execute

Description

An NCE operation executes the task instruction only if the condition register is false. The task instruction immediately following an NCE is always executed unless the instruction is marked for conditional execution, and the condition register is set accordingly.

The condition register (ctl.cnd) is contained within the controller unit. The condition register is tested at the start of the task execution instruction (e.g., the condition reflects the results of previous μ -task instructions) for the NCE operation.

The NCE operation causes the remaining operations in the task instruction to be executed only if the condition register is set appropriately. If the task instruction is predicated upon this operation, none of the other operations in the task instruction are completed (e.g., no result registers are modified).

Reference

Core Unit (Branch Control).

Syntax

NCE ;

Example

NCE ;



The following operations cannot be used simultaneously with any of the branch control instructions: DINT, EINT, PUSH or POP.

NEQ

Relational Not Equal

Description

A NEQ operation specifies a 32-bit comparison operation. If the signed 32-bit L.operand is equal to the signed 32-bit R.operand, the result is *1*; otherwise it is *0*.

Reference

Computational Unit (ALU_32).

Syntax

NEQ Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0 | *uname.r1* | *uname.r2* | *uname.r3*

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | **zero**

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies *1* for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the NEQ operation, no saturation is performed.

Cmask:

0x10

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

NEQ al0.r1 mp0.r0 sh0.r2 F T CL NS 0x10 ;

OR

Or (Bitwise)

Description

The OR operation is a bit wise 32-bit operation generating a 32-bit result.

Reference

Computational Unit (ALU_32).

Syntax

OR Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0 | *uname.r1* | *uname.r2* | *uname.r3*

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | **zero**

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register | **4-bit immediate**

The R.operand field specifies the right input data value, and may be a 32-bit global register or a 4-bit immediate (zero extended to 32 bits).

L.comp:

T | **F**

The L.comp field specifies whether a bit-wise compliment is applied to the input values. For the OR operation, the left input can be complimented (T) or not complimented (F).

R.comp:

T | **F**

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the OR operation, the right input can be complimented (T) or not complimented (F).

CarryIn:

CL | **CO**

The CarryIn field specifies the input carry value for addition operations. The value in this field is ignored for logical operations.

Saturate:

NS

The Saturate field specifies the kind of saturation that is performed. For the XOR operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the OR operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

OR al0.r0 al0.r3 0x0 F F CL NS ctl.cst1 ;

POP

Pop Top of Stack to Register

Description

A POP operation moves all 32-bit values within the hardware stack toward the top of the stack, removing the 32-bit value at the top of the stack.

Reference

Control Unit (overlaid with Branch Control).

Syntax

POP ;

Example

POP ;

*PR_INI

Program Register Initialize

Description

A PR_INI operation writes 32-bit immediate values to the program register. The PR_INI comprises an entire task instruction, and initializes as many 32-bit values as the instruction width supports.

Reference

Core Unit (Program Register).

Syntax

PR_INI data1 datan ;

Operand

Datan:

32-bit immediate value

The Datan field specifies the values to be stored in the constant registers **cstn**, where *n* is a value from one (1) through thirty (30).

Example

```
PR_INI 0x0 0x11111111 0x22222222 0x33333333 0x44444444 0x55555555 ;
```



The PR_INI is a special instruction that uses the entire VLIW and cannot be used simultaneously with any other special instruction or micro operation.

PR_ROT

Program Register Rotate

Description

A PR_ROT operation moves each value in the 32-bit program register to the right, while the value in program register 0 is shifted out.

Reference

Core Unit (Program Register).

Syntax

PR_ROT ;

Example

PR_ROT ;

PR_WR

Program Register Write

Description

A PR_WR operation moves each 32-bit program register value to the right while the value in program register 0 is replaced by the source register and loaded into the most significant program register.

Reference

Core Unit (Program Register).

Syntax

PR_WR Destination Source ;

Operand

Destination:

ctl.pr

The Destination field specifies the destination register of the result. It must be control program register.

Source:

32-bit register

The Source field specifies the input data to be shifted. For a PR_WR operation, the source data is always a 32-bit global register.

Example

```
PR_WR ctl.pr al0.r0 ;
```



The following operations cannot be used simultaneously with any of the program register instructions: CTD, CTS, SBR, SCR, SCTL or VPM.

The PR_WR operation uses the shared global bus and may conflict with other operations that use this resource.

PUSH

Push Top of Stack

Description

A PUSH operation moves all 32-bit values within the hardware stack toward the bottom of the stack, adding the register value to the 32-bit top of the stack.

Reference

Control Unit (overlaid with Branch Control).

Syntax

PUSH source ;

Operand

Source:

32-bit register

The Source field specifies the input data to be shifted. For a PUSH operation, the source data is always a 32-bit global register.

Example

```
POP al0.r0 ;
```



The following operations cannot be used simultaneously with the PUSH operation: BR, CALL, CE, DINT, EINT, ET, INTR, NBR, NCE, POP, RET, RETE, UBR or UET. The PUSH operation is only available to processors configured with a hardware stack.

R16S

Read 16-Bit Sign Extended

Description

An R16S operation specifies that the 16-bit value in memory at the specified address is read, sign extended to 32 bits, and loaded into the 32-bit read register. The 16 bits of data read is always aligned with the lower half of the read register.

Reference

Memory Interface Unit.

Syntax

R16S [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

mname.mport.rn | zero

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the R16S operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource.

If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

R16U

Read 16-Bit Unsigned

Description

An R16U operation specifies that the 16-bit value in memory at the specified address is read, zero extended to 32 bits, and loaded into the 32-bit read register. The 16 bits of data read is always aligned with the lower half of the read register.

Reference

Memory Interface Unit.

Syntax

R16U [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

***mname.mport.rn* | zero**

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the R16U operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource.

If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

R32

Read 32-Bit

Description

An R32 operation specifies that the memory word at the specified address is loaded into the read register.

Reference

Memory Interface Unit.

Syntax

R32 [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

***mname.mport.rn* | zero**

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the R32 operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource. If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

R8S

Read 8-Bit Sign Extended

Description

An R8S operation specifies that the 8-bit value in memory at the specified address is read, sign extended to 32 bits, and loaded into the 32-bit read register. The 8 bits of data read is always aligned with the lower quarter of the read register.

Reference

Memory Interface Unit.

Syntax

R8S [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

***mname.mport.rn* | zero**

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the R8S operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource. If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

R8U

Read 8-Bit Unsigned

Description

An R8U operation specifies that the 8-bit value in memory at the specified address is read, zero extended to 32 bits, and loaded into the 32-bit read register. The 8 bits of data read is always aligned with the lower quarter of the read register.

Reference

Memory Interface Unit.

Syntax

R8U [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

***mname.mport.rn* | zero**

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the R8U operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource.

If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

RET

Return From Sub-Routine

Description

The RET operation causes the program counter to be updated with the contents of the top of the stack, and removes or POPs the address from the stack. The instruction immediately following a RET is always executed, unless the instruction is marked for conditional execution and the condition register is set accordingly.

Reference

Core Unit (Branch Control).

Syntax

RET ;

Example

RET ;

RETE

Return From Interrupt Sub-Routine

Description

The RETE operation causes the program counter to be updated with the contents of the top of the stack. It also clears the interrupt mask register and removes or POPs the address from the stack. The instruction immediately following a RETE is always executed, unless the instruction is marked for conditional execution and the condition register is set accordingly.

A RETE operation clears the hardware interrupt mask.

Reference

Core Unit (Branch Control).

Syntax

RETE ;

Example

RETE ;

RPT

Repeat Next Instruction

Description

An RPT operation causes the instruction following the current instruction to be executed once and then repeated the number of times specified by the Count operand.

Reference

Core Unit (Repeat Counter).

Syntax

RPT Count ;

Operand

Count:

8-bit immediate

The Count field specifies the number of times the instruction is repeated. For an RPT operation, the instruction is always repeated to an 8-bit immediate value.

Example

RPT 0x6 ;



A repeated instruction (the instruction following an RPT operation) cannot be interrupted.

SBR

Set Branch Register

Description

An SBR operation updates the branch address register. A task instruction may contain one set branch register operation.

Reference

Core Unit (Branch Register).

Syntax

SBR Source ;

Operand

Source:

32-bit register | constant value

The Source field specifies the value for the branch address register. The lower 12 bits of a constant value or register are loaded into the branch address register.

Example

```
SBR 0x1033 PassFunction ;
```

SCR

Set Condition Register

Description

An SCR operation sets the condition register to true if any selected positive conditions are true, or if any selected negative conditions are false. In all other cases, the condition register is set to false.

An instruction containing an Set Counter Register (CTS) operation must also have an SCR operation that sets the condition register to !TC. An instruction containing a Decrement Counter Register (CTD) operation must also have an SCR operation that sets the condition register to TC or !TC.

Reference

Core Unit (Condition Register).

Syntax

SCR Source ;

Operand

Source:

constant value

The Source field specifies the condition mask for assignment of the condition register. The values for the Source field correspond to the following condition masks:

Condition Mask															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pol	-	cnd	zr	ng	co	ov	tc	-	-	!cnd	!zr	!ng	!co	!ov	!tc

Table 5-1. defines common values for the condition mask.

Table 5-1. Condition as values

Label	Value	Label	Value
EQ	0x1000	CO	0x0400
NEQ	0x0010	OV	0x2200
GT	0x9800	TC	0x0100
LT	0x0800	TRUE	0x2020
GTE	0x0008	1	0x2020
LTE	0x1800	!ZR	0x9000
UGT	0x9004	!NG	0x8800
ULT	0x0004	!CO	0x8400
UGTE	0x0400	!OV	0xA200
ULTE	0x1004	!TC	0x8100
ZR	0x1000	FALSE	0xA020
NG	0x0800	0	0xA020

Example

SCR 0x2020 ;

<-- set true on Terminal Count

SCS

Set Condition State

Description

An SCS operation can copy any global register, constant immediate, or constant data register into the *condition state* registers (`ctl.cs = {ctl.tc, ctl.cnd}`). These registers are the terminal count register of the counter and the condition register of the processor.

Reference

Core Unit (Counter Register).

Syntax

SCS Destination Source ;

Operand

Destination:

ctl.cs

The Destination field must be one of the condition state registers.

Source:

constant value | 32-bit register

The Source field specifies the input (set) data value, and can be any global register or constant value.

Example

```
SCS ctl.cs sh0.r1 ;
```

SCTL

Set Control Register

Description

An SCTL operation updates the Control Register.

Reference

Core Unit (Control Register).

Syntax

SCTL Source ;

Operand

Source:

32-bit register | constant value

The Source field specifies the value for the control register. The lower 12 bits of a constant value or register are loaded into the branch address register.

Example

```
SCTL a10.r0 ;
```


SEL

Select

Description

A SEL operation selects either the 32 most significant bits or the 32 least significant bits from the 64-bit Source, and places them in the least significant 32 bits of the 64-bit Destination.

If the Amount is 0, the 32 least significant bits are selected from the Source. If the Amount is not 0, the 32 most significant bits are selected from the Source.

Reference

Computational Unit (SHIFT_32).

Syntax

SEL Destination Source Amount Rounding Saturate ;

Operand

Destination:

uname.r1:uname.r0 | uname.r3:uname.r2

The Destination field must be two of the result registers within the shift unit.

Source:

32-bit register:32-bit register | 0:32-bit register | 0

The Source field specifies the input data to be shifted. For a SEL operation, the source data is always considered to be a 32-bit unsigned value. It is a concatenated 32-bit global register specification or zero.

Amount:

constant value | 32-bit register | zero

The Amount field specifies the number of bits the data is shifted. The least significant 6 bits (unsigned) of a register or constant are used. The shift amount must be in the range of 0 to 63.

Rounding:

NR

The Rounding field specifies whether that the result is not rounded (NR).

Saturate:

NS

The Saturate field specifies the type of saturation (if any) that is performed. For the SEL operation, no saturation is performed.

Example

```
SEL sh0.r1:sh0.r0 zero:a10.r2 0x1f NR NS ;
```

SETA

Set Address

Description

A SETA operation updates only the address registers defined for the Dest.ar and Src.ar fields in the primary MOP slot.

Reference

Memory Interface Unit.

Syntax

SETA [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

mname.mport.rn | zero

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the SETA operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource.

If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

SETWA

Set Address

Description

A SETWA operation updates only the address registers defined for the Dest.ar and Src.ar fields in the secondary MOP slot.

Reference

Memory Interface Unit.

Syntax

SWA [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

mname.mport.rn | zero

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the SETWA operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource.

If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

SFR

Set Flag Registers

Description

An SFR operation copies the contents of the global register into one of the flag registers in the ALU. The flag registers are required at the end of interrupt service routines to restore the processor state.

Reference

Computational Unit (ALU_32).

Syntax

SFR Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.status

The Destination field specifies the status of the ALU flag. The status can be ZR (zero), NG (negative), CO (carry out), or OV (overflow).

L.operand:

32-bit register | zero

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register | 4-bit immediate

The R.operand field specifies the right input data value, and may be a 32-bit global register or a 4-bit immediate (zero extended to 32 bits).

L.comp:

F

The L.comp field specifies that the left input is not complimented.

R.comp:

T | F | N | P

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the SFR operation, the right input can be complimented (T), not complimented (F), complimented if it is negative (N), or complimented if it is positive (P).

CarryIn:

CL | CO

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented, and 0 otherwise. CO specifies that the input carry value is the carry register, which is set to the carry out generated by the same arithmetic unit in the previous cycle. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

S8 | S16 | S32 | U8 | U16 | U32 | NS

The Saturate field specifies the kind of saturation that is performed. For saturation with an SFR operation, the source operands are always assumed to be signed, even when saturating to an unsigned result. Saturation is supported to a result width smaller than the source operands.

Cmask:

ctl.cst1

The Cmask field is not used in the SFR operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

```
SFR al0.status al0.r3 0x0 F F CL NS ctl.cst1 ;
```

SINTM

Set Interrupt Mask

Description

A SINTM operation sets the interrupt mask register, permitting or preventing acknowledgment of queued interrupt tasks. When the register is set to *0*, the interrupt mask is enabled. When the register is set to *1*, the interrupt mask is disabled.

Reference

Control Unit (overlaid with Branch Control).

Syntax

SINTM ;

Example

SINTM ;



The following operations cannot be used simultaneously with the SINTM operation: BR, CALL, CE, EINT, ET, INTR, NBR, NCE, PUSH, POP, RET, RETE, UBR or UET.

TQ

Task Queue

Description

The QT operation inserts a μ -task into the standard task queue of a task engine. The standard task queue can store the number of tasks configured by the user. When a task queue is full, queue task instructions specifying the engine containing the full queue will have no effect (e.g., the task to be queued is discarded and lost).

The Q-Bus is a shared resource among all engines, so it is possible that a higher priority μ -task engine has control of the Q-Bus when another engine also has a μ -task queue operation to execute. In this case, the lower priority engine may need to stall, thereby ceasing all program execution until the Q-Bus is available. There are buffers in place to limit the occurrence of stalled engines.

Reference

Core Unit (Task Queue).

Syntax

TQ Task.addr Parameter ;

Operand

Task.addr:

constant value | 32-bit register

The Task.addr field specifies the address of the μ -task being queued. The upper four (4) bits of the address specify the task engine, IO engine, or IO module. The lower 12 bits of the task.addr field specify the local task address in instruction memory.

Parameter:

constant value | 32-bit register

The Parameter field specifies a 16-bit parameter that is queued with the μ -task. The parameter value is loaded into the lowest number read address registers of the engine (pdm.rax0, sml.rax0, and smr.rap0) when the task is initiated.

Example

TQ 0xf00e ctl.cst1 ;



The TQ operation cannot be used simultaneously with any private data memory WX port write operation.

When the TQ operation uses a 32-bit register for the Task.addr operand, the shared global bus must be used which may conflict with other operations that use this resource.

TRUE

Relational Always

Description

A TRUE operation always sets the 32-bit result to 1.

Reference

Computational Unit (ALU_32).

Syntax

TRUE Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0 | uname.r1 | uname.r2 | uname.r3

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

zero

The L.operand field specifies the left input data value, and may be the constant zero.

R.operand:

zero

The R.operand field specifies the right input data value, and may be the constant zero.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation that is performed. For the TRUE operation, no saturation is performed.

Cmask:

0x2020

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

TRUE al0.r1 zero zero F T CL NS 0x2020 ;

UBR

Unconditional Branch

Description

A UBR operation unconditionally jumps to the branch address. The task instruction immediately following a UBR is always executed unless the instruction is marked for conditional execution, and the condition register is set accordingly.

The UBR operation causes the program counter to be updated with the content of the branch address register (ctl.br) regardless of the status of the condition register. If the μ -task instruction also contains a Set Branch Register (SBR) instruction, the program counter is updated directly with the new branch register value if the branch is taken.

Reference

Core Unit (Branch Control).

Syntax

UBR ;

Example

UBR ;

UET

Unconditional End Task

Description

A UET operation ends the current μ -task unconditionally without regard to the status of the condition register. The task instruction immediately following a UET is always executed unless the instruction is marked for conditional execution, and the condition register is set accordingly.

The condition register (ctl.cnd) is contained within the controller unit. The condition register is tested at the start of the task execution instruction (e.g., the condition reflects the results of previous μ -task instructions) for the UET operation.

The UET operation causes the next μ -task from the task queue to be initiated unconditionally without regard to the status of the condition register.

Reference

Core Unit (Branch Control).

Syntax

UET ;

Example

UET ;

UGT

Relational Unsigned Greater Than

Description

A UGT operation specifies a 32-bit comparison operation. If the unsigned 32-bit L.operand is greater than the unsigned 32-bit R.operand, the result is 1; otherwise it is 0.

Reference

Computational Unit (ALU_32).

Syntax

UGT Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the UGT operation, no saturation is performed.

Cmask:

0x9004

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

UGT al0.r1 mp0.r0 sh0.r2 F T CL NS 0x9004 ;

UGTE

Relational Unsigned Greater Than or Equal To

Description

A UGTE operation specifies a 32-bit comparison operation. If the unsigned 32-bit L.operand is greater than or equal to the unsigned 32-bit R.operand, the result is *1*; otherwise it is *0*.

Reference

Computational Unit (ALU_32).

Syntax

UGTE Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies *1* for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the UGTE operation, no saturation is performed.

Cmask:

0x400

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

UGTE al0.r1 mp0.r0 sh0.r2 F T CL NS 0x400 ;

ULT

Relational Unsigned Less Than

Description

A ULT operation specifies a 32-bit comparison operation. If the unsigned 32-bit L.operand is less than the unsigned 32-bit R.operand, the result is 1; otherwise it is 0.

Reference

Computational Unit (ALU_32).

Syntax

ULT Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the ULT operation, no saturation is performed.

Cmask:

0x4

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

ULT al0.r1 mp0.r0 sh0.r2 F T CL NS 0x4 ;

ULTE

Relational Unsigned Less Than or Equal

Description

A ULTE operation specifies a 32-bit comparison operation. If the unsigned 32-bit L.operand is less than or equal to the unsigned 32-bit R.operand, the result is 1; otherwise it is 0.

Reference

Computational Unit (ALU_32).

Syntax

ULTE Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. The value must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the 32-bit global register for the left input data value. The value must be a 15-bit signed magnitude input or zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

F

The L.comp field specifies that the left input is not complimented (F).

R.comp:

T

The R.comp field specifies that the right input is complimented (T).

CarryIn:

CL

The CarryIn field specifies the input carry value for addition operations. CL specifies 1 for carry in if the right operand is complemented. If no arithmetic operation is performed in a given cycle, the carry register retains the value from the previous cycle. Although AL0 generates a carry out flag, each arithmetic unit has its own independent carry register.

Saturate:

NS

The Saturate field specifies the kind of saturation performed. For the ULTE operation, no saturation is performed.

Cmask:

0x1004

The Cmask field specifies a constant value to be used for condition evaluation. If the same instruction contains a Set Condition Register (CTS) operation, this field must match the source field of the μ -OP.

Example

```
ULTE al0.r1 mp0.r0 sh0.r2 F T CL NS 0x1004 ;
```

W16

Write 16-Bit Value to Memory

Description

A W16 operation specifies that a 16-bit data value (src.ar) is written to memory.

Reference

Memory Interface Unit.

Syntax

W16 [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

mname.mport.rn | zero

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the W16 operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource.

If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

W32

Write 32-Bit Value to Memory

Description

A W32 operation specifies that a 32-bit data value (src.ar) is written to memory.

Reference

Memory Interface Unit.

Syntax

W32 [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

mname.mport.rn | zero

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the W32 operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource.

If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

W8

Write 8-Bit Value to Memory

Description

A W8 operation specifies that an 8-bit data value (src.ar) is written to memory.

Reference

Memory Interface Unit.

Syntax

W8 [Port] Rslt Dest.ar Src.ar Inc [Bitrev] [Wrap];

Operand

Port:

The Port field specifies the memory port name. This field is optional.

Rslt:

mname.mport.rn | zero

The Rslt field specifies the destination read register. It must be one of the read registers in the private or shared memory units.

Dest.ar:

address register | zero

The Dest.ar field specifies the destination address register. It must be zero or one of the address registers associated with the read port. If the dest.ar field is not zero, the sum of the address plus the increment value (inc) is loaded into this address register. The value in this field must be the same as src.ar.

Src.ar:

address register | zero

The Src.ar field specifies the source of the address value. It may be an address register associated with the read port, or 0.

Inc:

32-bit register | constant value | zero

The Inc field specifies the source of the increment value. It may be the lower 16 bits of a global 32-bit register, a constant value, or 0.

Bitrev:

bitrev

The keyword *bitrev* indicating that the increment value will be bit reversed.

Wrap:

wrap

The keyword *wrap* indicating that the address calculation will be wrapped.

Example



When the W8 operation uses a 32-bit register for the Inc operand, the shared global bus must be used which may conflict with other operations that use this resource. If local address registers are specified for the dest.ar and src.ar fields, they must share the same register number.

WPR

Write Program Register

Description

A WPR operation writes the contents of the program register to a line of instruction memory. The program register is stored in the instruction memory at the address specified by *al0.r0*.

Reference

Core Unit (overlaid with Counter).

Syntax

WPR ;

Example

WPR ;

XOR

Xor (Bitwise)

Description

The XOR operation is a bit wise 32-bit operation generating a 32-bit result.

Reference

Computational Unit (ALU_32).

Syntax

XOR Destination L.operand R.operand L.comp R.comp CarryIn Saturate Cmask ;

Operand

Destination:

uname.r0* | *uname.r1* | *uname.r2* | *uname.r3

The Destination field specifies the destination register of the result. It must be one of the result registers within the arithmetic unit.

L.operand:

32-bit register | zero

The L.operand field specifies the left input data value, and may be a 32-bit global register or the constant zero.

R.operand:

32-bit register

The R.operand field specifies the right input data value, and may be a 32-bit global register.

L.comp:

T | F

The L.comp field specifies whether a bit-wise compliment is applied to the input values. For the XOR operation, the left input can be complimented (T) or not complimented (F).

R.comp:

T | F

The R.comp field specifies whether a bit-wise compliment is applied to the input values. For the XOR operation, the right input can be complimented (T) or not complimented (F).

CarryIn:

CL | CO

The CarryIn field specifies the input carry value for addition operations. The value in this field is ignored for logical operations.

Saturate:

NS

The Saturate field specifies the kind of saturation that is performed. For the XOR operation, no saturation is performed.

Cmask:

ctl.cst1

The Cmask field is not used in the XOR operation. The field is used by the CND micro operation, and is needed by the assembler to generate the syntax for other ALU_32 instructions.

Example

```
XOR al0.r0 pdm.rx.r0 al0.r3 F F CL NS ctl.cst1 ;
```

Additional Help

For additional information or assistance, contact:

Application	Online Help and Readme text file
Phone	(978) 927-0555
Fax	(978) 927-0999
Internet	http://www.improvsys.com
Email	• info@improvsys.com • sales@improvsys.com • techsupport@improvsys.com
Postal	Improv Systems, Inc. 100 Cummings Street, Suite 343G Beverly, MA 01915